



GRADO EN INGENIERÍA ROBÓTICA SOFTWARE

Escuela de Ingeniería de Fuenlabrada

Curso académico 2023-2024

Trabajo Fin de Grado

Localización global de robots móviles en entornos desafiantes
mediante algoritmos bioinspirados

Tutor: Julio Salvador Lora Millán

Cotutor: Juan Carballeira López

Autor: Gonzalo Vega Pérez

Resumen

El presente trabajo aborda una de las principales problemáticas y pilar fundamental de la robótica móvil, la autolocalización. Centrándose principalmente en la localización en entornos interiores desafiantes.

La capacidad de determinar la posición y orientación de un robot es esencial para poder realizar tareas de navegación, mapeo y planificación de trayectorias. Sin embargo, esta tarea se vuelve especialmente compleja en entornos difíciles como las minas, en los que la odometría es poco confiable debido al error acumulativo causado por el deslizamiento de las ruedas sobre un terreno inestable y factores como la baja visibilidad y el aislamiento con el exterior imposibilitan el uso algunos de los principales sensores de la industria como los sistemas GPS y las cámaras RGBD.

En este contexto, a diferencia de los métodos tradicionales de localización que hacen uso de la fusión sensorial, se aborda el desafío de la autolocalización precisa y robusta utilizando un LiDAR como único sensor.

Para abordar esta problemática, se han implementado varios algoritmos evolutivos en un nodo de ROS2, que, posteriormente, han sido evaluados utilizando distintos conjuntos de datos y mediante simulación utilizando Gazebo.

Los experimentos realizados para evaluar los algoritmos implementados mostraron resultados prometedores. Se observó una alta tasa de convergencia utilizando únicamente datos del LiDAR, lo que demuestra su eficacia en entornos donde la fusión sensorial no es una opción. Además, se encontró que los algoritmos evolutivos mantienen una eficiencia computacional aceptable, lo que los hace adecuados para implementarse a bordo de diversos sistemas robóticos.

Acrónimos

GPS *Global Positioning System*

RGBD *Red Green Blue Depth*

ROS2 *Robot Operating System 2*

LiDAR *Light Detection and Ranging*

KF *Kalman Filter*

EKF *Extended Kalman Filter*

MCL *Monte Carlo Localization*

PF *Particle Filter*

GA *Genetic Algorithm*

DE *Differential Evolution*

PSO *Particle Swarm Optimization*

IWO *Invasive Weed Optimization*

CSV *Comma-Separated Values*

Índice general

1. Introducción	1
1.1. Introducción a la autolocalización en robots móviles	1
1.2. Autolocalización en interiores	2
1.3. Autolocalización en entornos desafiantes	3
1.4. Autolocalización con algoritmos evolutivos	3
2. Objetivos	5
2.1. Descripción del problema	5
2.2. Requisitos de desarrollo	5
2.3. Metodología	6
3. Plataforma de desarrollo	7
3.1. Hardware	7
3.2. Implementación de los algoritmos evolutivos	7
3.3. Integración en ROS2 y evaluación	7
4. Diseño	9
4.1. Funcionamiento básico del nodo de ROS2	9
4.2. Funcionamiento del nodo en simulación	10
4.3. Implementación de los algoritmos evolutivos	12
4.3.1. Inicialización	13
4.3.2. Evaluación	13
4.3.3. Reproducción y reemplazo	13
4.3.4. Condiciones de parada	13
4.4. Características del algoritmo Differential Evolution	14
4.5. Características del algoritmo Particle Swarm Optimization	16
4.5.1. Actualización de la velocidad	17
4.5.2. Actualización de la posición	18
4.6. Características del algoritmo Invasive Weed Optimization	18

4.6.1. Proceso de creación de nuevas plantas	19
4.7. Herramientas de análisis de datos	21
5. Resultados y Discusión	22
5.1. Resultados	22
5.1.1. Dificultad del escenario	23
5.1.2. Porcentaje de convergencia	24
5.1.3. Tiempo de ejecución y número de iteraciones	25
5.1.4. Errores en posición y orientación	28
5.2. Resultados de simulación	34
5.2.1. Dificultad del escenario de simulación	36
5.2.2. Porcentaje de convergencia en simulación	37
5.2.3. Tiempo de ejecución y número de iteraciones en simulación . . .	38
5.2.4. Errores en posición y orientación en simulación	39
5.3. Discusión	41
6. Conclusiones	44
6.1. Análisis de los objetivos propuestos	44
6.2. Líneas de trabajos futuros	44
6.3. Competencias	45
Bibliografía	46

Índice de figuras

4.1. Robot Turtlebot3 Burger de Robotis Co.	11
4.2. Sensor LiDAR Velodyne VLP-16	11
5.1. Nube de puntos global del conjunto de datos. Recuadrado en azul el área de operación del robot al capturar las 44 nubes de puntos.	23
5.2. Ubicaciones centrales de cada nube de puntos local dentro del entorno.	23
5.3. Porcentaje de convergencia por nube de puntos	25
5.4. Tiempo de ejecución por nube de puntos	27
5.5. Número de iteraciones por nube de puntos	28
5.6. Error de posición en ejecuciones exitosas por nube de puntos respecto al umbral de convergencia (línea punteada).	30
5.7. Error de orientación en cabeceo por nube de puntos respecto al umbral de convergencia (línea punteada).	31
5.8. Error de orientación en alabeo por nube de puntos respecto al umbral de convergencia (línea punteada).	32
5.9. Error de orientación en guiñada en ejecuciones exitosas por nube de puntos respecto al umbral de convergencia (línea punteada).	33
5.10. Nube de puntos global del entorno de simulación	34
5.11. Posiciones elegidas para las pruebas en el entorno de simulación	35
5.12. Porcentaje de convergencia por nube de puntos en simulación	37
5.13. Tiempo de ejecución por nube de puntos en simulación	38
5.14. Número de iteraciones por nube de puntos en simulación	39
5.15. Error de posición en ejecuciones exitosas por nube de puntos en simulación respecto al umbral de convergencia (línea punteada)	39
5.16. Error de orientación en cabeceo y alabeo por nube de puntos en simulación respecto al umbral de convergencia (línea punteada)	40
5.17. Error de orientación en guiñada en ejecuciones exitosas por nube de puntos en simulación respecto al umbral de convergencia (línea punteada)	41

Listado de ecuaciones

4.1. Función de mutación de DE	14
4.2. Índice de reparto de error	15
4.3. Actualización de la velocidad en PSO	17
4.4. Actualización de la posición en PSO	18
4.5. Cálculo del número de semillas en IWO	19
4.6. Actualización de sigma en IWO	20

Índice de cuadros

5.1. Resumen de los resultados principales	42
5.2. Resumen de los fallos en convergencia	42

Capítulo 1

Introducción

En este capítulo se explorará el estado actual de la autolocalización en robots móviles. Centrándonos en la localización en entornos interiores donde la pose inicial es desconocida. Se abordarán desde los enfoques tradicionales basados en la fusión de sensores hasta las últimas innovaciones utilizadas en entornos desafiantes como los algoritmos evolutivos.

1.1. Introducción a la autolocalización en robots móviles

La autolocalización en robots móviles se ha convertido en uno de los temas de mayor importancia en el sector de la robótica debido a la proliferación de estos sistemas tanto en el mercado de la robótica de servicios como en el de la robótica industrial. En particular, los Autonomous Mobile Robots (AMRs) están desempeñando un papel crucial en la automatización de tareas en almacenes, fábricas y otros entornos dinámicos ([Li et al., 2024b]). Si bien los robots móviles en exteriores pueden hacer uso del GPS para la localización, la situación es diferente en interiores, donde las señales de satélite son inaccesibles o poco confiables. ([Yarovi and Cho, 2024])

Basándonos en la información de la posición inicial, podemos clasificar el problema de localización en tres categorías: seguimiento de posición, posicionamiento/localización global, y problema del robot secuestrado ([Panigrahi and Bisoy, 2022]).

En el caso del problema de seguimiento de posición, la posición inicial del robot es conocida. El objetivo es estimar de manera precisa la pose (posición y orientación) del robot en cada instante de tiempo según se mueve por el entorno. Dos de las técnicas probabilísticas más comunes para resolver este problema son el filtro de Kalman extendido (EKF) y el filtro de partículas (PF) ([Negenborn, 2003]).

Por otra parte, en los escenarios de posicionamiento global y en el problema del robot secuestrado la pose inicial del robot es desconocida. Esto significa que el robot debe ser capaz de localizarse a sí mismo en un mapa global haciendo uso de los datos obtenidos de sus sensores. El primero de los escenarios se da cuando el robot se enciende o cuando la pose del robot se actualiza a un punto arbitrario del mapa mientras se realiza el seguimiento de la posición. En cambio, el problema del robot secuestrado se refiere a una situación en la que el robot es reubicado físicamente a otra parte del entorno por un agente externo. El robot debe ser capaz de detectar que el entorno a su alrededor ha cambiado de manera imprevista e iniciar una maniobra de autolocalización para estimar su nueva pose dentro del mapa ([Panigrahi and Bisoy, 2022]).

1.2. Autolocalización en interiores

En un escenario típico de localización de robots en interior se dispone de un mapa global del entorno. Además, el robot está equipado con sensores que observan este entorno y que monitorizan su propio movimiento. El problema de la localización se convierte entonces en uno de estimar la posición y orientación del robot dentro del mapa utilizando la información recopilada por estos sensores. Las técnicas de localización de robots deben ser capaces de manejar observaciones ruidosas y generar no solo una estimación de la ubicación del robot sino también una medida de la incertidumbre de la estimación de la ubicación. ([Huang and Dissanayake, 2016]).

Los enfoques tradicionales de autolocalización en robots móviles se fundamentan en técnicas bien establecidas como la odometría y la triangulación. La odometría se basa en la medición del desplazamiento del robot mediante sensores integrados en sus ruedas. Estos sensores registran el giro de las ruedas del robot, lo que, junto con la medida del radio de las mismas, permite calcular una estimación de la distancia recorrida a partir de la integración de estos datos. Si bien la odometría puede proporcionar estimaciones de posición en tiempo real, su precisión tiende a degradarse con el tiempo debido a errores acumulativos causados por el deslizamiento de las ruedas o la irregularidad del terreno, entre otros factores ([Belter and Nowicki, 2019]).

Por otro lado, la triangulación implica la utilización de señales de referencia, como balizas o imágenes de referencia conocidas colocadas en lugares estratégicos del entorno, para determinar la posición del robot en relación con estos puntos. Este método se basa en la medición de la distancia y el ángulo entre el robot y las señales de referencia

utilizando técnicas de visión artificial y geométricas para calcular su posición relativa. Si bien la triangulación puede proporcionar una mayor precisión que la odometría en entornos específicos, su eficacia puede verse limitada por la necesidad de disponer de señales de referencia visibles y la susceptibilidad a errores causados por la interferencia o la obstrucción de estas señales ([Font and Batlle, 2006]) ([Nemec et al., 2019]).

1.3. Autolocalización en entornos desafiantes

Un entorno desafiante es aquel que presenta dificultades que complican la realización de las tareas por parte de un robot. Por ejemplo, una mina presenta un gran número de dificultades para el robot a la hora de autolocalizarse: debido al aislamiento con el exterior, el uso de GPS es inviable; el uso de cámaras RGBD presenta grandes dificultades debido a la baja luminosidad y suciedad del ambiente; debido a la inestabilidad del terreno, los datos de odometría son poco fiables y no sirven para autolocalizarse; y además, dependiendo del tamaño de los túneles, colocar balizas de referencia puede ser muy ineficiente ([Carballeira López, 2022]).

En líneas generales, los entornos desafiantes suelen presentar restricciones en la cantidad de sensores disponibles para el robot. Debido a ello, cada vez son más comunes las investigaciones y desarrollos de técnicas que posibilitan la autolocalización prescindiendo de la fusión sensorial. Entre estas técnicas se incluyen los algoritmos evolutivos, los cuales posibilitan que el robot se autolocalice empleando exclusivamente un sensor LiDAR.

1.4. Autolocalización con algoritmos evolutivos

La computación evolutiva se deriva de la simulación de la selección natural y los procesos genéticos en la evolución biológica ([Li et al., 2024a]). En las últimas dos décadas se han publicado diversas investigaciones que compartían el objetivo de aplicar distintos algoritmos evolutivos a las técnicas de localización tradicionales como el filtro de Kalman extendido (EKF), el algoritmo de agrupación de Markov (MCL) y el filtro de partículas (PF). Algunas de las principales técnicas evolutivas estudiadas son la optimización por enjambre de partículas (PSO) ([Havangi, 2019], [Havangi et al., 2010], [Pinto et al., 2015], [Dastjerdi et al., 2016]), los algoritmos genéticos (GA) ([Moreno et al., 2002], [Rajurkar et al., 2016]), y la evolución diferencial (DE) ([Huo et al., 2013]).

La aplicación de estos algoritmos ha evidenciado mejorar la robustez y flexibilidad de los sistemas. Por ejemplo, de acuerdo a los hallazgos presentados en [Pinto et al., 2015], se ha logrado un margen de error de 0,04 metros y 2 grados en simulaciones, así como de 0,06 metros y 7 grados en escenarios reales.

En este trabajo se estudiarán tres algoritmos metaheurísticos bioinspirados de manera aislada: Differential Evolution (DE), Particle Swarm Optimization (PSO) e Invasive Weed Optimization (IWO). Los dos primeros algoritmos, DE y PSO, fueron presentados en 1995, son ampliamente conocidos y han demostrado ser robustos y eficaces en diversas aplicaciones de optimización. El algoritmo DE es conocido por su capacidad para manejar problemas de alta dimensionalidad y su sencilla implementación, lo que facilita su uso en una gran variedad de problemas de optimización ([Yang et al., 2007]). Por otro lado, PSO se basa en el comportamiento social de los enjambres de insectos, los bancos de peces o las bandadas de pájaros. Ha sido utilizado en numerosos campos debido a su rápida convergencia y capacidad para evitar mínimos locales mediante técnicas híbridas que combinan PSO con otros métodos, como el simplex de Nelder-Mead, para mejorar la eficiencia y precisión del algoritmo ([Steffen, 2022]).

El tercer algoritmo, Invasive Weed Optimization (IWO), fue presentado por primera vez en 2006. En comparación con DE y PSO es un algoritmo menos probado, pero presenta características interesantes que lo diferencian de otros algoritmos. IWO se inspira en el proceso natural de colonización de las malas hierbas, y ha mostrado potencial en la optimización de funciones no lineales y multimodales ([Sur and Shukla, 2013]). Su capacidad para explorar el espacio de búsqueda de manera eficaz y su mecanismo adaptativo para ajustar la distribución de soluciones lo hacen una opción prometedora para la localización de robots móviles en entornos desafiantes.

Evaluar estos tres algoritmos en el contexto de la autolocalización de robots permitirá no solo comprobar la eficacia de los métodos más establecidos, sino también explorar las capacidades y limitaciones de IWO en comparación con los enfoques más tradicionales.

En los próximos capítulos se explorará en detalle los tres algoritmos evolutivos, requisitos de diseño, implementación y finalmente los resultados obtenidos.

Capítulo 2

Objetivos

En este capítulo, se abordarán los principales objetivos de este estudio, así como los requisitos propuestos para poder realizar un análisis correcto y la metodología empleada durante el desarrollo.

2.1. Descripción del problema

El objetivo principal de este trabajo es desarrollar una herramienta que pueda ser ejecutada a bordo de un robot real y permita evaluar los tres algoritmos evolutivos mencionados en el capítulo anterior para determinar su eficacia como herramienta de autolocalización de un robot en un entorno desafiante. Es relevante señalar que la situación a evaluar representa uno de los mayores retos para la localización de un robot, dado que se cuenta únicamente con una nube de puntos adquirida mediante un sensor LiDAR, y el robot no puede moverse ni girar para obtener más información del entorno. Evaluar los algoritmos en esta situación tan desafiante es crucial, ya que permite conocer los límites y las verdaderas capacidades de cada uno.

2.2. Requisitos de desarrollo

Para obtener resultados concluyentes es necesario seguir una serie de requisitos específicos. Para poder verificar el correcto funcionamiento de los algoritmos en simulación, es crucial asegurar que el robot esté correctamente simulado, sea incapaz de moverse y que disponga únicamente de un sensor LiDAR, que debe estar integrado con ROS2 adecuadamente. Además, es fundamental que la pose real del robot, proporcionada por el simulador, sea precisa para poder compararla con la pose calculada por el algoritmo y así determinar el error cometido. Con el fin de realizar un análisis estadístico riguroso y profundo de los resultados obtenidos, los algoritmos deberán ser ejecutados por cada nube de puntos del conjunto de datos un mínimo de diez veces. En base a los datos presentados en [Pinto et al., 2015], para considerar una

ejecución como exitosa se ha establecido un umbral de error de 0,1 metros en posición y 8 grados en cada eje de rotación. Aunque el tiempo de convergencia también es un factor importante a considerar, no se ha fijado un umbral debido a que el límite admisible para este parámetro puede variar significativamente dependiendo de la aplicación en la que se utilicen estos algoritmos.

2.3. Metodología

El desarrollo del trabajo se ha organizado en tres etapas principales: la implementación de los algoritmos, su integración en ROS2 y la fase de evaluación. Sin embargo, es importante destacar que las dos primeras etapas no han seguido una progresión lineal. Inicialmente, se implementó y se integró un solo algoritmo en ROS2 antes de proceder con los otros dos. Este enfoque permitió una mayor eficiencia en el proceso, asegurando un funcionamiento sólido y eficaz del software desarrollado.

Durante el desarrollo se siguió una metodología con objetivos cortos y concretos, dando gran importancia al correcto funcionamiento del software implementado antes de pasar de etapa. La primera meta fue la implementación del algoritmo Differential Evolution en Python, partiendo de la implementación de este algoritmo en MatLab. Una vez verificado el correcto funcionamiento del algoritmo comparando los resultados obtenidos de ambas implementaciones, se integró en un nodo de ROS2. Tras evaluar satisfactoriamente el comportamiento del nodo, se realizó lo mismo con los otros dos algoritmos: IWO y PSO.

Finalmente, para realizar la evaluación de los resultados obtenidos, se desarrollaron varias herramientas en Python que permiten agilizar el proceso de análisis de los datos, así como generar gráficas y figuras que faciliten el entendimiento de los mismos.

Capítulo 3

Plataforma de desarrollo

En este capítulo se tratará el hardware y software utilizados en cada una de las tres etapas de desarrollo.

3.1. Hardware

Todas las partes del trabajo han sido desarrolladas y evaluadas en un hardware común, un ordenador de sobremesa con hardware comercial. Las principales especificaciones del equipo utilizado son:

- Tarjeta gráfica: nVidia GTX 970 4GB GDDR5
- Procesador: Ryzen 2700 8 núcleos a 3.20 GHz
- Memoria RAM: 16 GB DDR4 a 3200 MHz

3.2. Implementación de los algoritmos evolutivos

La implementación inicial de los algoritmos se llevó a cabo en un sistema Windows 11 23H2 utilizando Matlab R2023b para el análisis inicial de los algoritmos y Python 3.11 para el desarrollo de los mismos. Para la visualización de los resultados obtenidos se utilizó la librería open3d de Python, que permite representar nubes de puntos de gran tamaño en tres dimensiones.

3.3. Integración en ROS2 y evaluación

Una vez implementados los algoritmos se cambió el sistema operativo de desarrollo a Ubuntu 22.04 debido a sus facilidades para el desarrollo software e integración con ROS2. Se utilizó la versión Humble de ROS2 y la versión 3.10.12 de Python.

Se creó un nodo interactivo de ROS2 que incluye los tres algoritmos y permite al usuario seleccionar un entorno de evaluación, así como el algoritmo a utilizar y sus

parámetros. Además, permite elegir entre ejecutar el algoritmo seleccionado en tiempo real, es decir, leyendo los datos actuales de un LiDAR publicados en un topic de ROS2 por un robot simulado o utilizar nubes de puntos que fueron obtenidas previamente y guardadas en formato PLY. El nodo también cuenta con un modo automático que permite ejecutar el algoritmo seleccionado en un bucle continuo, explorando cada una de las nubes locales almacenadas hasta que el usuario decida detenerlo. Este modo demostró ser de gran utilidad, ya que permitió al sistema funcionar de forma autónoma durante largas sesiones, generando así una cantidad considerable de datos.

Para la visualización de los resultados se prescindió del uso de la biblioteca open3d de Python a favor de publicar las nubes de puntos en dos topics de ROS2: uno para el mapa global y otro para el mapa local obtenido tras la ejecución del algoritmo. Este enfoque posibilita el uso de herramientas como RVIZ para visualizar los resultados obtenidos. Por otra parte, los resultados numéricos, así como toda la información relevante obtenida durante la ejecución del algoritmo, son mostrados al usuario por la terminal y guardados en un documento CSV para su posterior evaluación.

Para el análisis de los datos almacenados en el fichero CSV se desarrollaron varias herramientas auxiliares que permiten analizar los datos haciendo uso de diversas bibliotecas de Python para procesamiento de datos, estadística y creación de gráficas.

Capítulo 4

Diseño

En este capítulo se tratará detalladamente cada uno de los tres algoritmos implementados de manera separada. Se analizarán primero desde un punto de vista teórico para después analizar sus respectivas implementaciones en código. También se explicará el funcionamiento general del nodo de ROS2, su funcionamiento en modo offline y con un robot simulado y las herramientas de análisis de datos desarrolladas.

4.1. Funcionamiento básico del nodo de ROS2

Los tres algoritmos han sido implementados en un mismo nodo de ROS2 que funciona de manera aislada, es decir, no requiere de ningún otro nodo para funcionar. Los dos requisitos fundamentales del algoritmo son: una nube de puntos global del entorno y un conjunto de nubes de puntos locales. El nodo cuenta con dos parámetros de configuración: 'auto' y 'simulated'. El parámetro 'auto' permite ejecutar el nodo en bucle utilizando una nube local diferente por cada ejecución, mientras que el parámetro 'simulated' permite ejecutar el algoritmo seleccionado sobre un robot real o simulado. La lógica de funcionamiento del nodo se puede resumir en cuatro pasos:

1. **Pedir al usuario los parámetros de ejecución:**

- a) **Nube de puntos como escáner local.** Se pide seleccionar una de las nubes locales del subconjunto mencionado previamente. Los algoritmos se aplicarán sobre esta nube para obtener su pose estimada.
- b) **Tipo de algoritmo.** Permite seleccionar cuál de los tres algoritmos se quiere aplicar.
- c) **Parámetros del algoritmo.** Se debe especificar el tamaño de la población y el número máximo de iteraciones.

2. **Ejecutar algoritmo seleccionado:**

El objetivo del algoritmo es obtener una estimación de la pose de la nube local a partir de la nube global. Puesto que se está trabajando en un entorno

tridimensional, la pose se compone por seis valores, una traslación y una rotación por cada uno de los ejes X, Y y Z. El algoritmo finaliza una vez se cumple una de las condiciones de convergencia o si se alcanza el número de iteraciones máximo especificado.

3. Mostrar resultados:

Una vez finalizado el algoritmo se muestra la pose estimada y el error cometido, así como otros parámetros de interés obtenidos durante la ejecución del algoritmo, como el tiempo de ejecución. Para poder visualizar los resultados primero se calcula la transformación espacial que se debe aplicar a la nube local a partir de la pose estimada. Por último se publica la nube global en el topic `/map_global`, la nube local transformada en el topic `/map_local` y se guardan los datos en un fichero `.csv`. Si se ha conseguido una buena estimación, ambas nubes aparecerán superpuestas.

Si el nodo se ha ejecutado estableciendo el valor del parámetro `auto` a `True`, tras finalizar el paso tres se volverá a ejecutar el nodo desde el paso dos cambiando la nube de puntos local por la siguiente del conjunto.

4.2. Funcionamiento del nodo en simulación

Al cambiar el parámetro `simulated` a `True`, el nodo puede operar con un robot real o simulado, leyendo las nubes de puntos de un topic según son publicadas por el robot.

Para probar esta funcionalidad, se han utilizado dos herramientas de software libre diferentes. El paquete `Multi-turtlebot3-Gazebo-ROS2` permite simular en `Gazebo` un robot `Turtlebot3 Burger` equipado con un sensor `LiDAR Velodyne VLP-16`, que puede ser teleoperado utilizando el teclado. Por otro lado, el paquete `lidarslam_ros2` permite realizar SLAM en el entorno utilizando un `LiDAR` y generar un mapa global, el cual es almacenado en un directorio accesible por el nodo.

La principal diferencia respecto al modo de funcionamiento `offline` es que no es necesario solicitar al usuario la nube de puntos local sobre la que se aplicará el algoritmo. En su lugar, el nodo obtiene esta información suscribiéndose al topic `/velodyne_points`, generado por `Multi-turtlebot3-Gazebo-ROS2`, donde se publica la nube de puntos local continuamente. Para obtener la pose real del robot en simulación y calcular el error cometido, el nodo crea un segundo suscriptor al topic `/odom`, generado

por el simulador Gazebo, donde se publica, entre otros datos, la pose actual del robot simulado.

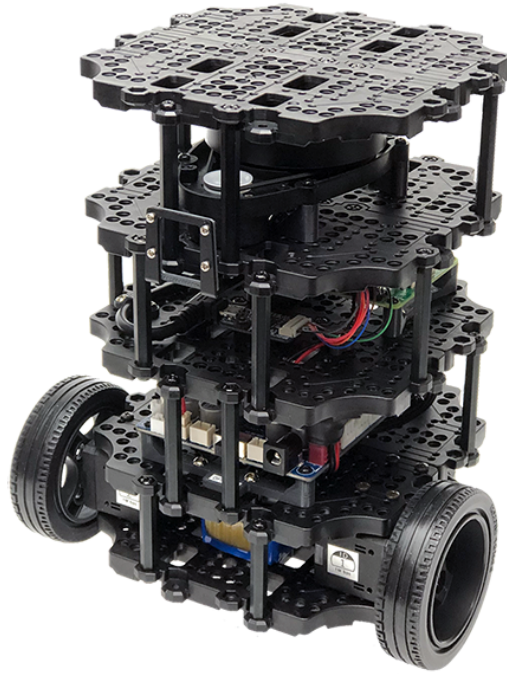


Figura 4.1: Robot Turtlebot3 Burger de Robotis Co.



Figura 4.2: Sensor LiDAR Velodyne VLP-16

4.3. Implementación de los algoritmos evolutivos

Los algoritmos evolutivos se basan en el concepto de candidatos. Un candidato se caracteriza por seis parámetros (las posiciones y rotaciones en cada eje) y un coste, que es directamente proporcional al error respecto al objetivo. Un conjunto de candidatos forman una población.

El algoritmo evalúa cada candidato y le asigna un coste. Después, descarta los peores y genera una nueva población a partir de los mejores, este proceso se denomina iteración. La función matemática que determina como se genera la siguiente población varía dependiendo del algoritmo y es el factor de mayor relevancia a la hora de determinar el comportamiento del mismo. En los tres algoritmos implementados esta función matemática se inspira en fenómenos biológicos naturales, por ello hablamos de algoritmos bioinspirados.

El objetivo del algoritmo es encontrar al candidato óptimo. Es decir, el candidato que minimiza el error en las seis variables y por tanto, el que tiene el menor coste asociado.

Para optimizar la búsqueda, es esencial alcanzar un equilibrio adecuado entre exploración y explotación. La exploración implica la capacidad de generar candidatos alejados entre sí, cubriendo diferentes regiones del espacio de búsqueda, lo que facilita encontrar soluciones diversas y evitar quedar atrapado en mínimos locales. Por otro lado, la explotación se centra en mejorar las soluciones existentes al enfocarse en áreas específicas del espacio de búsqueda donde se encuentran candidatos prometedores. En la práctica, el algoritmo debe ser capaz de explorar lo suficiente para descubrir soluciones potenciales y luego explotar esas soluciones para refinarlas.

En general todos los algoritmos evolutivos se dividen en seis etapas.

- Inicialización
- Evaluación
- Reproducción
- Evaluación
- Reemplazo

- Condiciones de parada

Las cuatro últimas etapas forman el bucle principal del algoritmo, que se ejecutará continuamente hasta que se cumpla una condición de parada. A continuación se explica la lógica común a todos los algoritmos de cada una de las etapas.

4.3.1. Inicialización

El primer paso es generar una población con tantos candidatos como haya seleccionado el usuario. El primer candidato se inicializa en el origen, mientras que el resto se generan con valores aleatorios dentro de los límites definidos por el mapa local. A continuación se evalúa cada uno de los candidatos para asignarles un coste.

4.3.2. Evaluación

Para calcular el coste de cada candidato, primero se aplica una transformación espacial a la nube de puntos local utilizando la pose del candidato. A continuación se define una región de interés dentro de la nube global utilizando los límites de la nube local. Una vez obtenidas las dos nubes de puntos del mismo tamaño, se crea una matriz de correspondencia para asociar cada punto de la nube local con su vecino más cercano dentro de la nube global recortada. Por último se calcula la distancia euclídea entre cada punto de la nube local y su vecino asociado y se utiliza el error cuadrático medio como función de coste para determinar el coste del candidato.

4.3.3. Reproducción y reemplazo

La reproducción es el primer paso del bucle principal del algoritmo y el más importante. En general, por cada candidato se genera uno nuevo utilizando la función matemática correspondiente a cada algoritmo y se evalúa nuevamente. Si el coste del nuevo candidato es menor que el actual, el candidato actual es reemplazado.

En los puntos posteriores se explicará en detalle las funciones utilizadas por cada uno de los algoritmos.

4.3.4. Condiciones de parada

Se verifica si se han cumplido alguna de las condiciones de parada: convergencia del algoritmo o número máximo de iteraciones alcanzado. De ser así, el algoritmo termina y se devuelve el mejor candidato encontrado.

Los condiciones de convergencia son las siguientes:

- **Convergencia total:** Todos los candidatos tienen el mismo coste. Implica que se ha convergido a una solución única y no se están generando nuevas soluciones viables.
- **Convergencia normal:** El error promedio de la población y el coste del peor individuo entre el coste del mejor individuo son menores que un umbral.
- **Convergencia invariable:** Se ha superado el límite de iteraciones sin mejoras significativas. Se ha alcanzado un punto de convergencia invariable y el algoritmo puede detenerse

4.4. Características del algoritmo Differential Evolution

Differential Evolution (DE) presenta dos parámetros fundamentales que influyen significativamente en su comportamiento:

- **Factor de mutación (F):** Este parámetro controla la capacidad del algoritmo para explorar y explotar el espacio de búsqueda. Su valor típicamente varía entre 0 y 2, donde valores más altos promueven una exploración más amplia del espacio de búsqueda. El valor inicial optimizado empíricamente es de 0,9.
- **Tasa de cruce (CR):** Indica la probabilidad de que cada parámetro de un candidato mute, generando así un nuevo candidato a evaluar. El valor optimizado empíricamente es de 0,75.

La estrategia de reproducción en DE se basa en mutaciones aleatorias. Por cada candidato de la población se generará uno nuevo, n . Los parámetros del candidato que prevalecerá en la población, llamado candidato final, se asignarán en función de la tasa de cruce. Como la tasa de cruce ha sido establecida en 0,75, aproximadamente el 75 % de los parámetros del candidato final pertenecerán a n , mientras que el resto serán los del candidato original. La ecuación que define al nuevo candidato n es:

$$n = c + F \times (a - b) \quad (4.1)$$

Ecuación 4.1: Función de mutación de DE

Donde, a , b , y c son tres candidatos de la población seleccionados aleatoriamente y F es el factor de mutación. Un valor mayor de F llevará al candidato n más lejos de

los candidatos a , b , y c , lo que resulta en una exploración más amplia del espacio de búsqueda.

DE es un algoritmo adaptativo capaz de modificar su política de exploración y explotación según el estado actual de la búsqueda. Para determinar este estado, se estudian dos valores: el índice de reparto de error y la dispersión de los costes.

- **Dispersión de Costes:** Es la relación entre el peor y el mejor coste de la población. Una dispersión cercana a 1 indica una gran similitud entre los candidatos, sugiriendo convergencia.
- **Índice de Reparto de Error:** Evalúa cómo se distribuye el coste entre todos los candidatos de la población. Un valor más cercano a 1 indica un mayor homogeneidad entre los candidatos y por tanto, mayor convergencia. Su valor se calcula a partir de la siguiente función:

$$I = \frac{\sum_{i=0}^{NP} Coste_i}{NP \cdot MejorCoste} \quad (4.2)$$

Ecuación 4.2: Índice de reparto de error

Donde:

- I es el índice de reparto de error.
- NP es el número total de candidatos en la población.
- i es el índice que identifica a cada candidato dentro de la población.
- $Coste_i$ es el coste asociado al i -ésimo candidato.
- $MejorCoste$ es el valor del coste mínimo entre todos los candidatos de la población.

Ambas medidas permiten conocer el grado de convergencia actual de la población.

- **Convergencia Inicial:** Cuando el índice de reparto de error es menor que 2 y la dispersión de costes es menor que 2,5, la población ha comenzado a converger. En este caso, el factor de mutación se reduce a 0,7 para favorecer la explotación de una región específica del espacio de búsqueda.
- **Convergencia Significativa:** Cuando el índice de reparto de error es menor que 1,25 y la dispersión de costes es menor que 1,5, la convergencia es más

significativa. En este punto, el factor de mutación F se reduce a 0,3 para refinar aún más la solución. Además, el tamaño de la población se reduce a una quinta parte de su valor inicial, ya que la región del espacio de búsqueda que se está explorando es mucho más pequeña y no es necesario trabajar con una población tan grande para cubrir esa región.

4.5. Características del algoritmo Particle Swarm Optimization

Los candidatos del algoritmo PSO se denominan “partículas” y cuentan con varias características:

- **Velocidad:** Velocidad actual de la partícula
- **Posición:** Posición actual de la partícula
- **Mejor posición:** Mejor posición alcanzada por la partícula
- **Coste:** Coste actual de la partícula
- **Mejor coste:** Mejor coste alcanzado por la partícula

Además, la población, denominada “enjambre”, cuenta con dos propiedades:

- **Mejor posición global:** Mejor posición alcanzada entre todas las partículas
- **Mejor coste global:** Mejor coste alcanzado entre todas las partículas

Las partículas se desplazan por el espacio de búsqueda guiadas por su velocidad e influenciadas tanto por su mejor posición personal como por la mejor posición global encontrada por el enjambre. En cada iteración, se actualizan la posición y la velocidad de todas las partículas según dos funciones. La función de actualización de la posición depende de la posición y velocidad actuales de la partícula, mientras que la función de actualización de la velocidad se divide en tres componentes: la inercia, la atracción hacia la mejor posición personal y la atracción hacia la mejor posición global. El peso de cada uno de estos componentes puede ajustarse mediante los siguientes parámetros para modificar el comportamiento del algoritmo.

- c_1 : Coeficiente de aprendizaje cognitivo, determina cuánto se mueve la partícula hacia su mejor posición personal. Su valor optimizado empíricamente es 1,5.

- c_2 : Coeficiente de aprendizaje social, determina cuánto se mueve la partícula hacia la mejor posición global. Su valor optimizado empíricamente es 2.
- w : Factor de inercia, controla la influencia de la velocidad previa en la nueva velocidad de la partícula. Su valor optimizado empíricamente es 1.
- w_d : Factor de amortiguamiento de inercia, utilizado para disminuir gradualmente la inercia durante las iteraciones del algoritmo. Su valor optimizado empíricamente es 0,99.

El componente de mayor peso en el cálculo de la velocidad es la atracción global, ya que queremos que las partículas exploren alrededor de las regiones más prometedoras; seguido de la atracción personal, que permite a cada partícula refinar su solución y diferenciarse de las demás; y por último la inercia, que fomenta la exploración, ayudando a escapar de posibles mínimos locales. Cada iteración, el factor de inercia se actualiza multiplicando su valor por el factor de amortiguamiento de inercia. Al ser este menor que 1, la inercia disminuye progresivamente según avanza el algoritmo, provocando una mayor explotación de las regiones del espacio de búsqueda con soluciones prometedoras.

4.5.1. Actualización de la velocidad

La velocidad de cada partícula se actualiza cada iteración siguiendo la ecuación 4.3

$$v_i^{t+1} = w \cdot v_i^t + c_1 \cdot r_1 \cdot (p_i^{best} - x_i^t) + c_2 \cdot r_2 \cdot (g^{best} - x_i^t) \quad (4.3)$$

Ecuación 4.3: Actualización de la velocidad en PSO

Donde:

- $w \cdot v_i^t$ es el componente de inercia.
- $c_1 \cdot r_1 \cdot (p_i^{best} - x_i^t)$ es el componente de atracción personal.
- $c_2 \cdot r_2 \cdot (g^{best} - x_i^t)$ es el componente de atracción global.
- v_i^{t+1} es la nueva velocidad de la partícula i en el tiempo $t + 1$.
- v_i^t es la velocidad actual de la partícula i en el tiempo t .

- w es el factor de inercia que controla la influencia de la velocidad previa en la nueva velocidad.
- c_1 es el coeficiente de aprendizaje cognitivo, que determina cuánto se mueve la partícula hacia su mejor posición personal.
- c_2 es el coeficiente de aprendizaje social, que determina cuánto se mueve la partícula hacia la mejor posición global.
- r_1 y r_2 son valores aleatorios entre 0 y 1, generados independientemente en cada iteración.
- p_i^{best} es la mejor posición personal de la partícula i .
- g^{best} es la mejor posición global encontrada por el enjambre.
- x_i^t es la posición actual de la partícula i en el tiempo t .

Si la nueva posición de la partícula la situa fuera del espacio de búsqueda, se invierte el valor de la velocidad y se situa la partícula en el límite del espacio de búsqueda.

4.5.2. Actualización de la posición

La posición de cada partícula se actualiza cada iteración siguiendo la ecuación 4.4

$$x_i^{it+1} = x_i^{it} + v_i^{it} \quad (4.4)$$

Ecuación 4.4: Actualización de la posición en PSO

Donde:

- x_i^{it+1} es la nueva posición de la partícula i en la iteración $it + 1$.
- x_i^{it} es la posición actual de la partícula i en la iteración it .
- v_i^{it} es la velocidad actual de la partícula i en la iteración it .

4.6. Características del algoritmo Invasive Weed Optimization

IWO es un algoritmo inspirado en las malas hierbas en el que los candidatos, también denominados “plantas”, esparcen “semillas” por todo el espacio de búsqueda.

Al igual que en el algoritmo DE, cada planta se caracteriza únicamente por una posición y un coste. IWO cuenta con 5 parámetros que pueden ser modificados para ajustar su comportamiento:

- $S_{\min}$: Número mínimo de semillas generadas por una planta. Su valor optimizado empíricamente es 0.
- $S_{\max}$: Número máximo de semillas generadas por una planta. Su valor optimizado empíricamente es 4.
- σ_i : Desviación estándar inicial de la distribución normal que siguen las semillas. Su valor optimizado empíricamente es 0,5.
- σ_f : Desviación estándar final de la distribución normal que siguen las semillas. Su valor optimizado empíricamente es 0,001.
- **exp**: Parámetro que controla la tasa de cambio de σ . Su valor optimizado empíricamente es 2.

4.6.1. Proceso de creación de nuevas plantas

Determinación del número de semillas

El primer paso en el proceso de reproducción del algoritmo IWO es determinar el número de semillas que producirá cada planta de la población en una iteración determinada. Este número es proporcional al coste actual de cada planta. Las plantas con menor coste producirán un mayor número de semillas, siempre comprendido entre $S_{\min}$ y $S_{\max}$. La función utilizada para calcular el número de semillas es la siguiente:

$$S = S_{\min} + (S_{\max} - S_{\min}) \left(\frac{C_w - C_p}{C_w - C_b} \right) \quad (4.5)$$

Ecuación 4.5: Cálculo del número de semillas en IWO

Donde:

- S es el número de semillas calculado.
- $S_{\min}$ es el número mínimo de semillas permitido.
- $S_{\max}$ es el número máximo de semillas permitido.

- C_w es el peor coste de la población.
- C_p es el coste de la planta actual.
- C_b es el mejor coste de la población.

Dispersión de las semillas

Las semillas se generarán alrededor de la planta madre siguiendo una distribución normal. La desviación estándar de esta distribución, σ , decrece según progresa el algoritmo, lo que significa que las semillas se situarán cada vez más cerca de la planta madre, promoviendo la explotación del espacio de búsqueda. La desviación estándar de la distribución se actualiza cada iteración siguiendo la siguiente función:

$$\sigma = \left(\frac{\text{iter_max} - it}{\text{iter_max} - 1} \right)^{\text{exp}} (\sigma_i - \sigma_f) + \sigma_f \quad (4.6)$$

Ecuación 4.6: Actualización de sigma en IWO

Donde:

- σ es la desviación estándar calculada.
- $iter_max$ es el número máximo de iteraciones permitido.
- it es el número de iteraciones actual.
- $\exp$ es el parámetro que controla la tasa de cambio de σ .
- σ_i es la desviación estándar inicial.
- σ_f es la desviación estándar final.

La pose de cada semilla se calcula a partir de la pose de la planta madre y un vector aleatorio escalado por σ . Este vector se obtiene de una distribución normal y su magnitud decrece a medida que σ disminuye.

Exclusión competitiva

De cada una de las semillas generadas se creará una nueva planta. Para evitar el crecimiento exponencial de la población, se combinan la nueva población creada a partir de las semillas con la población original en una sola. La nueva población combinada se ordena por coste. Si su tamaño excede el tamaño máximo permitido, se eliminan las plantas con mayor coste hasta alcanzar el límite de población.

4.7. Herramientas de análisis de datos

Tras cada ejecución del nodo, los resultados se guardan en un fichero `.csv`. Este fichero contiene tantas columnas como parámetros se deseen almacenar, siendo los principales: identificador de la nube de puntos local, algoritmo seleccionado, tiempo transcurrido, número de iteraciones, error de posición y los tres errores en orientación en tres columnas diferentes. Cada ejecución del nodo añade una nueva fila al fichero.

Para visualizar y estudiar los datos recogidos en el archivo `.csv`, se ha creado un programa en Python que permite graficar los resultados de cada algoritmo utilizando las siguientes bibliotecas:

- **Pandas**: para acceder y manipular el archivo `.csv`.
- **Numpy**: para agrupar los datos y extraer información relevante.
- **Matplotlib**: para generar gráficas.

Capítulo 5

Resultados y Discusión

En este capítulo se analizarán los resultados obtenidos durante las pruebas de simulación de cada uno de los tres algoritmos. Para determinar la eficiencia y eficacia de cada algoritmo respecto a los demás se analizarán los siguientes parámetros: porcentaje de convergencia, tiempo de ejecución, número de iteraciones, error de posición y error de orientación.

5.1. Resultados

El conjunto de datos utilizado para probar los algoritmos fue obtenido escaneando con un sensor LiDAR montado sobre un robot móvil parte de un pasillo y un despacho. El conjunto se compone de 44 nubes, cada una formada por 262.144 puntos y un mapa global de 2.305.299 puntos formado a partir de todas ellas. Las primeras 30 nubes de puntos fueron tomadas en el pasillo, mientras que las últimas 14 se obtuvieron dentro del despacho, una habitación cuadrada y poco amueblada. Para minimizar el coste computacional durante la ejecución de los algoritmos, se realizó un submuestro aleatorio para reducir el número de puntos de las nubes locales a 2.622 y de la nube global a 46.106.

Los tres algoritmos se ejecutaron un mínimo de diez veces sobre cada nube de puntos del conjunto. Las nubes de puntos que mostraron resultados menos consistentes fueron sometidas a un mayor número de ejecuciones para un análisis más detallado. En total, se obtuvieron 491 resultados para el algoritmo DE, 465 para PSO y 455 para IWO, obteniendo un total de 1.411 resultados entre los tres algoritmos.

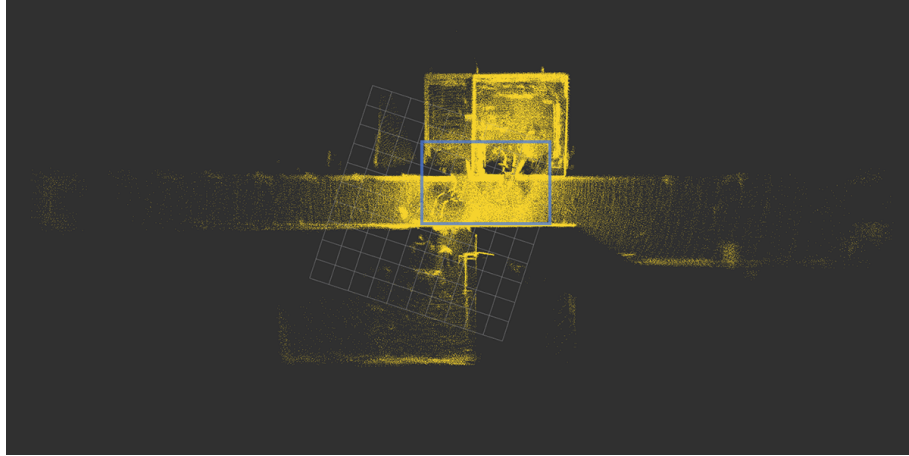


Figura 5.1: Nube de puntos global del conjunto de datos. Recuadrado en azul el área de operación del robot al capturar las 44 nubes de puntos.

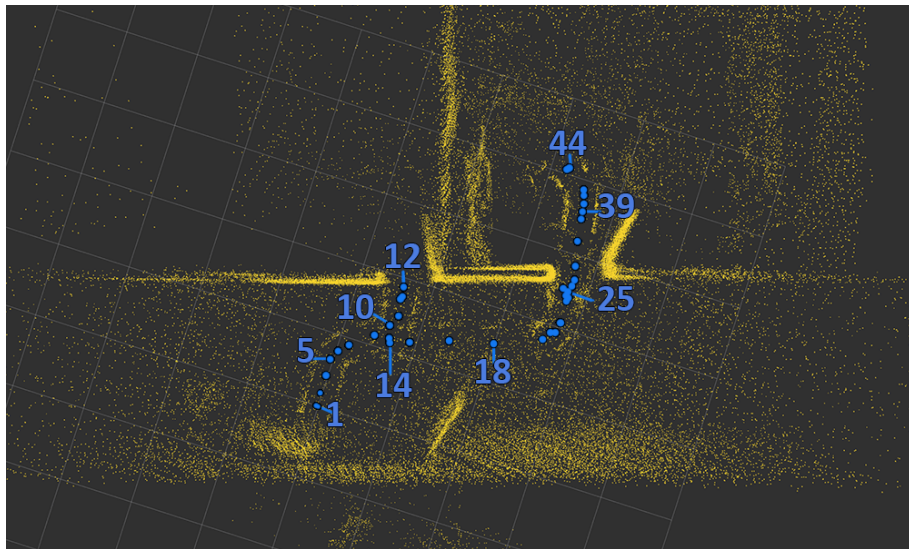


Figura 5.2: Ubicaciones centrales de cada nube de puntos local dentro del entorno.

5.1.1. Dificultad del escenario

La principal dificultad a la que se enfrentan los algoritmos radica en la falta de textura en el entorno. La textura se refiere a la abundancia de elementos representativos capturados en la nube de puntos, los cuales permiten distinguir diferentes regiones del entorno. Por ejemplo, en un túnel vacío con paredes lisas, la ausencia de elementos distintivos dificulta enormemente determinar el sentido de la orientación o la ubicación

exacta dentro del túnel. De igual manera, ciertas partes del pasillo y el despacho escaneados presentan un bajo nivel de textura y alto nivel de simetría, lo que las asemeja al problema del túnel y dificulta la autolocalización del robot.

5.1.2. Porcentaje de convergencia

El porcentaje de convergencia se refiere a la cantidad de veces que cada algoritmo es capaz de obtener un resultado satisfactorio por cada nube de puntos.

Los umbrales de error establecidos para considerar una ejecución como exitosa se basan en los resultados obtenidos durante las pruebas en escenarios realistas presentados en [Pinto et al., 2015]. El estudio se centra en el problema de seguimiento de posición utilizando el filtro de Kalman extendido y fusión sensorial. Sin embargo, cuando el error en la localización es alto, se utiliza el algoritmo PSO para realizar una autolocalización global. El error obtenido en escenarios realistas es de 0,06 metros y 7 grados. Debido a la naturaleza desafiante de los escenarios escogidos para la realización de las pruebas en el presente trabajo, los umbrales de error establecidos son ligeramente más altos. Los objetivos propuestos para considerar una ejecución como exitosa son los siguientes:

- **Iteraciones.** Se debe alcanzar la convergencia en menos de 500 iteraciones.
- **Error de posición.** Se debe obtener un error de posición menor a 0,1 metros.
- **Error de orientación.** Se debe obtener un error de orientación menor a ocho grados en cada eje.

Para considerar que un algoritmo ha funcionado correctamente al ejecutarse sobre una nube de puntos local, al menos el 80 % de las ejecuciones realizadas con esa nube deben haber obtenido un error en posición menor a 0,1 metros y un error en orientación en cada eje menor a ocho grados.

De las 44 nubes de puntos del conjunto, el algoritmo DE funcionó correctamente en 28 de ellas, PSO en 31 e IWO en 23. Lo que se traduce en unos porcentajes de éxito del 63,63 %, 70,45 % y 52,27 % respectivamente. Como se ve en la figura 5.3, los tres algoritmos presentan resultados similares, teniendo dificultades en las mismas nubes de puntos del conjunto: 3, 5, 6, 19, 20, 22 y las últimas 14, que fueron tomadas dentro

del despacho.

El algoritmo IWO presenta un comportamiento polarizado: en situaciones sencillas converge el 100 % de las veces, mientras que en situaciones difíciles su tasa de convergencia es inferior al 50 %. En contraste, el algoritmo PSO no alcanza el 100 % de convergencia en tantas ocasiones pero su media de convergencia es cercana al 70 % y se mantiene superior al 50 % en la mayoría de situaciones. Por su parte, el algoritmo DE presenta un comportamiento intermedio entre los dos.

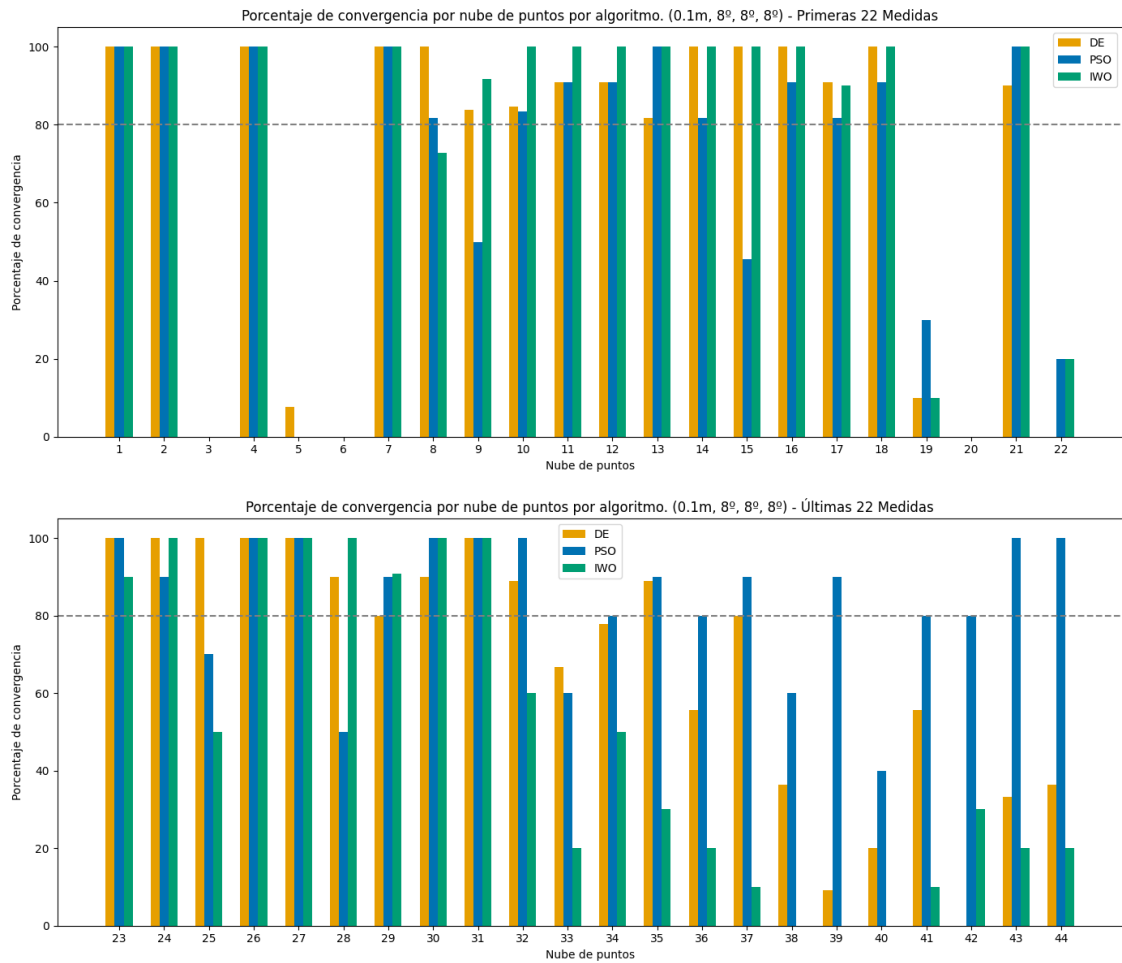


Figura 5.3: Porcentaje de convergencia por nube de puntos

5.1.3. Tiempo de ejecución y número de iteraciones

El tiempo de ejecución se refiere al tiempo transcurrido desde el comienzo de la primera iteración hasta haber alcanzado una condición de parada. Para medir este tiempo se utilizó la función *time* de la biblioteca *time* de Python, que devuelve los

segundos transcurridos desde el 1 de enero de 1970. Para obtener el tiempo de ejecución se obtienen los segundos transcurridos al comienzo y al final de la ejecución y se calcula su diferencia.

Las diferencias en el tiempo de ejecución de los algoritmos son significativas. DE presenta un tiempo de ejecución medio de 183 segundos, PSO de 599 segundos e IWO de 1.404 segundos. El tiempo de ejecución depende de dos factores: tiempo por iteración y número de iteraciones. El tiempo por iteración es similar en los tres algoritmos, alrededor de cinco segundos. Sin embargo, el número de iteraciones varía significativamente.

El algoritmo DE se detuvo en todas las ejecuciones tras 50 iteraciones, el número mínimo de iteraciones permitido, habiendo alcanzado un tipo de convergencia normal. Además es el único algoritmo que no ha detenido ninguna ejecución por haber llegado al límite de iteraciones máximas establecido

El algoritmo IWO, por el contrario, requiere de 300 iteraciones de media para converger. Comparando las figuras 5.3 y 5.5 se puede observar que las nubes de puntos que presentan peores resultados de convergencia para el algoritmo IWO son a la vez las que tienen un menor número de iteraciones de media. Al igual que el algoritmo DE, todas las convergencias alcanzadas fueron de tipo normal.

Por último, el algoritmo PSO se encuentra en un término medio entre DE e IWO, con 101 iteraciones de media y muy poca variabilidad entre las distintas nubes. Sin embargo, al igual que en el algoritmo IWO se puede observar cómo el número de iteraciones disminuye ligeramente en algunas de las nubes de puntos que presentan mayores dificultades para este algoritmo, principalmente las nubes 3, 5 y 6. De igual forma que los otros dos algoritmos, todas las convergencias fueron de tipo normal.

El hecho de que un algoritmo no converja exitosamente pero detenga la ejecución antes de haber alcanzado el máximo de iteraciones es un signo de que se ha quedado atrapado en un mínimo local; el algoritmo converge a una solución mejor que las cercanas, pero no a la mejor solución global. En esta situación, el algoritmo PSO ha demostrado ser el más robusto de los tres. Los algoritmos DE e IWO, por otro lado, evidencian una mayor tendencia a la convergencia prematura en situaciones difíciles que requieren mayor exploración.

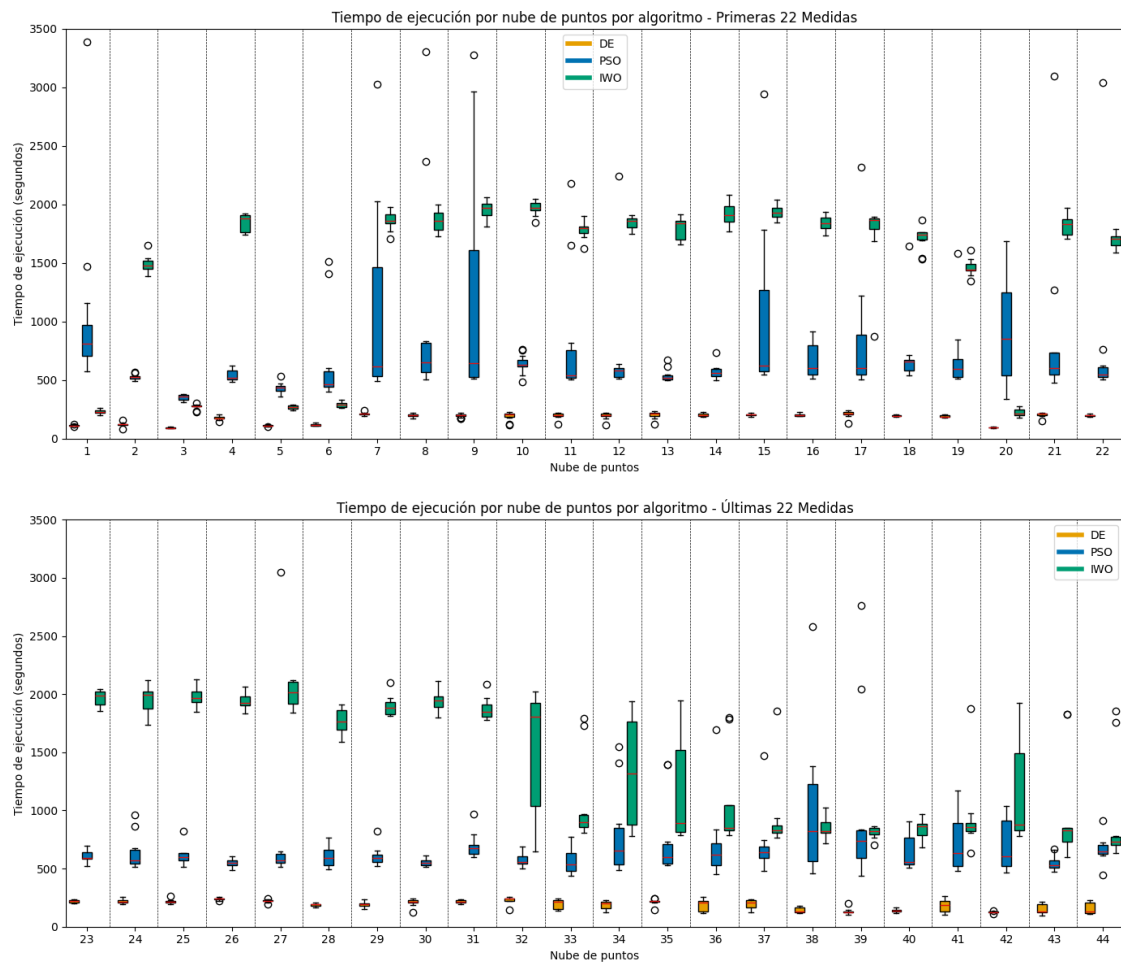


Figura 5.4: Tiempo de ejecución por nube de puntos

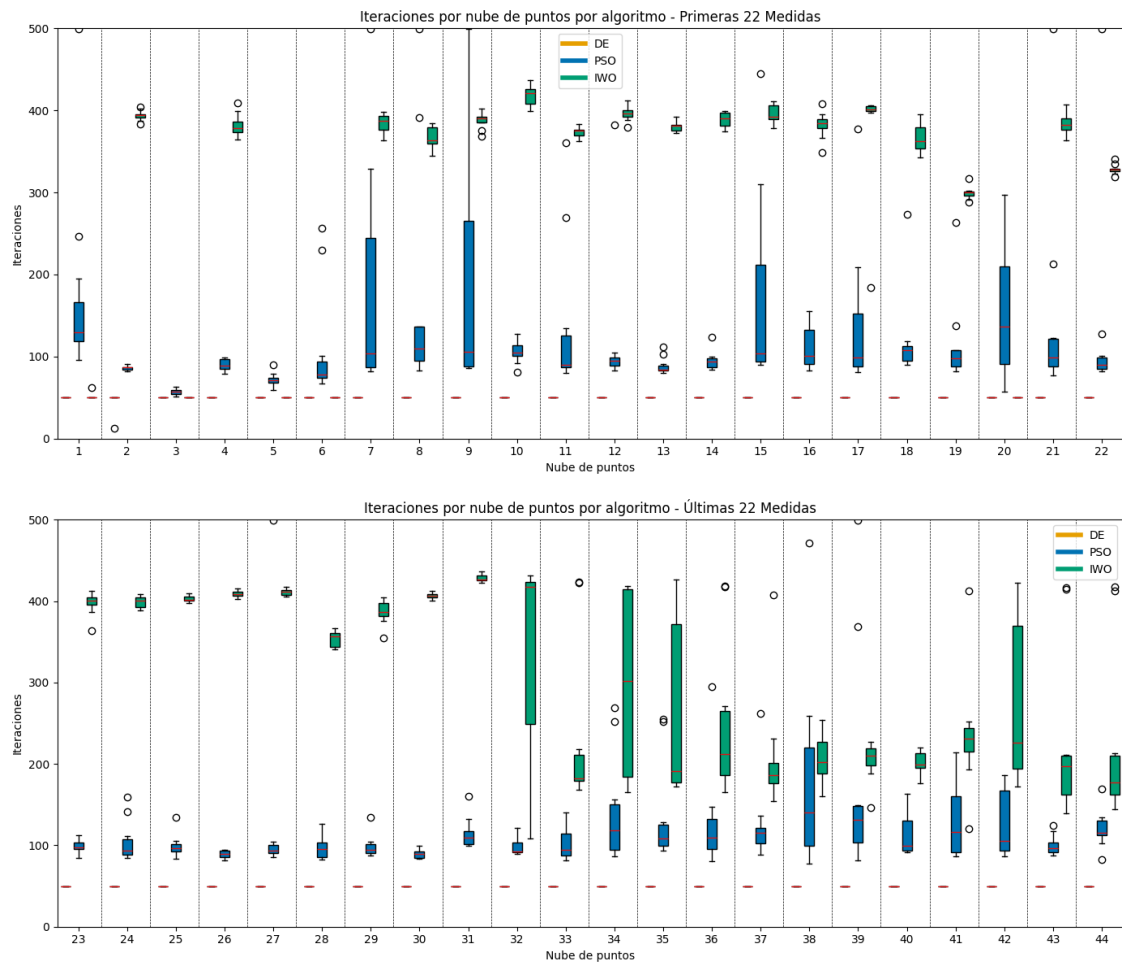


Figura 5.5: Número de iteraciones por nube de puntos

5.1.4. Errores en posición y orientación

Durante la ejecución de los algoritmos existen dos errores a minimizar: error en posición y error en orientación. Para el análisis de los datos, el error de posición se ha declarado como la distancia euclídea desde la posición calculada hasta la posición real, mientras que los errores en orientación han sido estudiados por separado.

Error en posición

Las nubes 3, 6, 20, 22 y 42 presentaron los peores resultados en posición, ningún algoritmo fue capaz de obtener un error menor a 0,1 metros en ninguna de estas nubes. Las cuatro primeras fueron capturadas en puntos del pasillo donde los únicos elementos distintivos, las puertas y el interior del despacho, a penas eran visibles, dificultando

la tarea de ubicar al robot a lo largo del mismo. Por otra parte, la nube 42 se tomó dentro del despacho. Las últimas 14 nubes se obtuvieron según el robot se acercaba al centro del despacho desde la puerta. Estas nubes de puntos presentan dificultades para los algoritmos porque las paredes laterales del despacho coinciden a lo largo de todo el recorrido y los únicos elementos representativos son la pared del fondo del despacho, el trozo de pasillo que queda atrás y los muebles del interior del despacho. Sin embargo, el ruido en las medidas hace que los muebles apenas sean visibles dentro de la nube.

Como se ve en la gráfica 5.6, el error en posición promedio en las ejecuciones exitosas se situó entre los 0,02 y los 0,04 metros. Sin embargo, en algunas nubes como la 27, 28 y 29, la distribución de las medidas fue más amplia, con errores comprendidos entre los 0,01 y 0,1 metros. Además, algunas de las nubes que presentaron un menor porcentaje de convergencia, principalmente las últimas 14, obtuvieron errores en posición cercanos a los 0,02 metros en las ejecuciones en las que tuvieron éxito. Es decir, aunque los algoritmos tendían a equivocarse un mayor número de veces en estas nubes, cuando convergían correctamente lo hacían con una alta precisión.

Del total de ejecuciones, DE produjo fallos en posición superiores al umbral el 30,14 % de ocasiones, PSO el 25,16 % e IWO el 38,46 %.

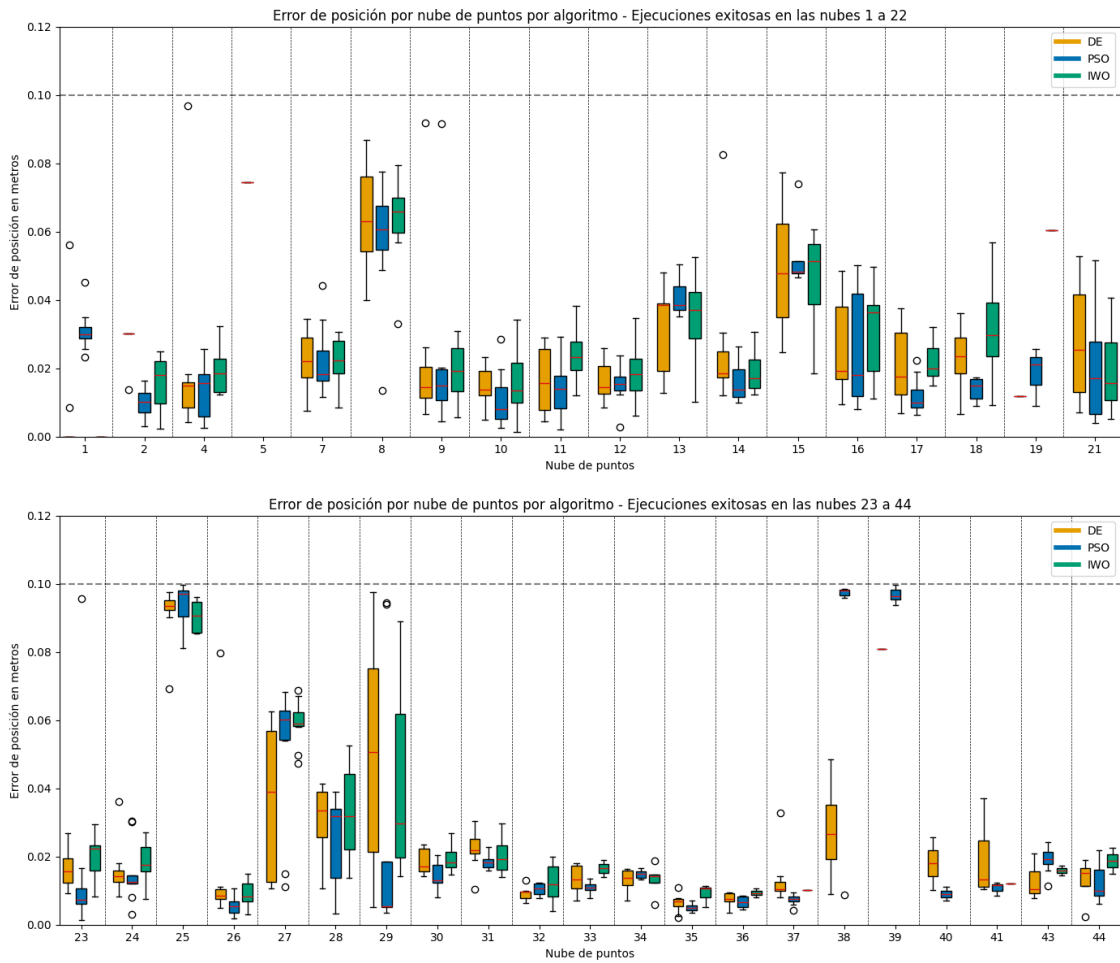


Figura 5.6: Error de posición en ejecuciones exitosas por nube de puntos respecto al umbral de convergencia (línea punteada).

Errores en cabeceo y alabeo

Los errores en cabeceo y alabeo nunca han superado los ocho grados, a excepción de la nube 41 para el algoritmo IWO. Debido a la naturaleza del robot, al escanear la escena, el robot mantuvo fijo estos dos ejes, moviéndose solo en guiñada. Debido a esto, un pequeño cambio en cualquiera de estos dos ejes alejaría rápidamente los puntos de la nube local de los de la nube global, provocando un aumento significativo en el error cometido y el coste del candidato, que probablemente, sería descartado.

Del total de ejecuciones, DE produjo errores en cabeceo superiores al umbral de ocho grados de error el 0,81 % de ejecuciones, PSO el 0,43 % e IWO el 7,69 %. Respecto al error en alabeo, DE obtuvo un error superior a ocho grados en una ejecución, lo que

supone el 0,2% del total de ejecuciones, mientras que PSO e IWO se mantuvieron por debajo del umbral en todas las ejecuciones.

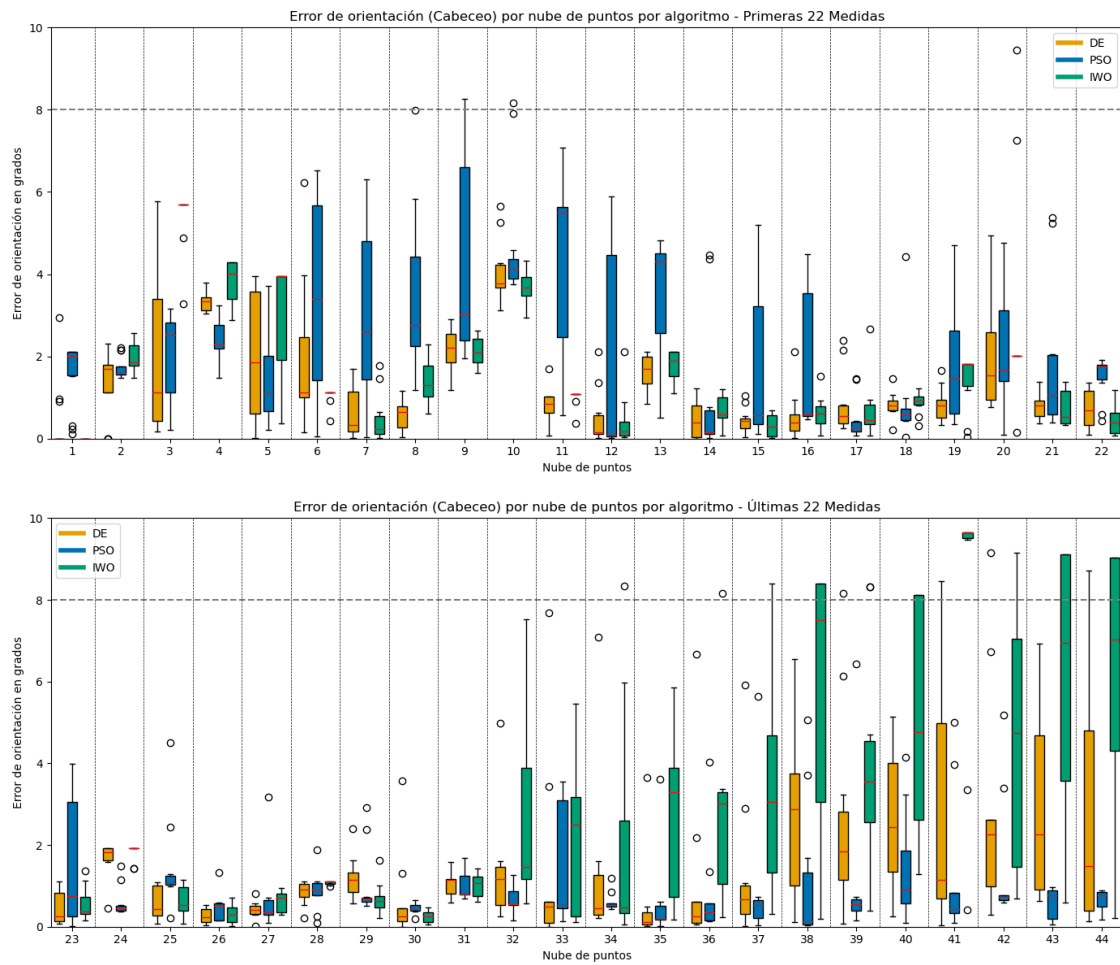


Figura 5.7: Error de orientación en cabeceo por nube de puntos respecto al umbral de convergencia (línea punteada).

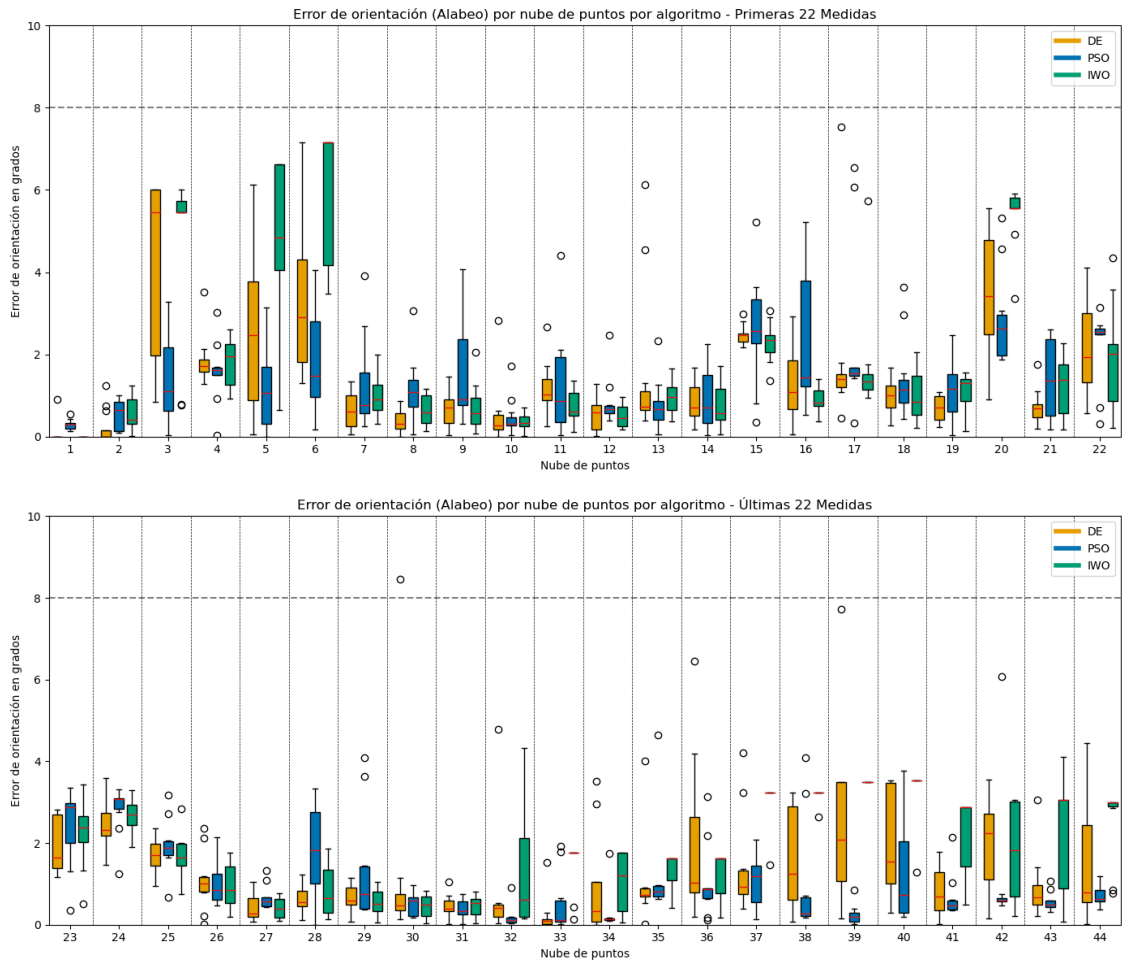


Figura 5.8: Error de orientación en alabeo por nube de puntos respecto al umbral de convergencia (línea punteada).

Error en guiñada

A diferencia del error en posición, en guiñada, los tres algoritmos consiguieron converger al menos una vez con un error menor al umbral en todas las nubes de puntos, a excepción de las nubes 38, 39 y 40 en las que el algoritmo IWO fue incapaz de converger con un error en guiñada menor a 8 grados.

Dentro de las ejecuciones exitosas, las nubes 5, 6 y 20 presentaron los peores resultados. Aunque los tres algoritmos se mantuvieron por debajo del umbral casi en la totalidad de las ejecuciones en estas nubes, la media de error se situó entre los 4 y los 7 grados de error, mientras que en el resto de escenarios la media fue inferior a los 2 grados de error.

En cuanto a las últimas 14 nubes de puntos, capturadas dentro del despacho, también se observan errores notables en guiñada. PSO fue el algoritmo más preciso, con errores superiores a ocho grados en solo una de las catorce nubes. DE, en cambio, presentó estos errores en cinco de ellas. Finalmente, IWO tuvo el peor desempeño, cometiendo errores en guiñada superiores a ocho grados en once de las catorce nubes de puntos.

Del total de ejecuciones, DE produjo fallos en guiñada superiores al umbral el 17,72 % de ocasiones, PSO el 10,54 % e IWO el 22,64 %.

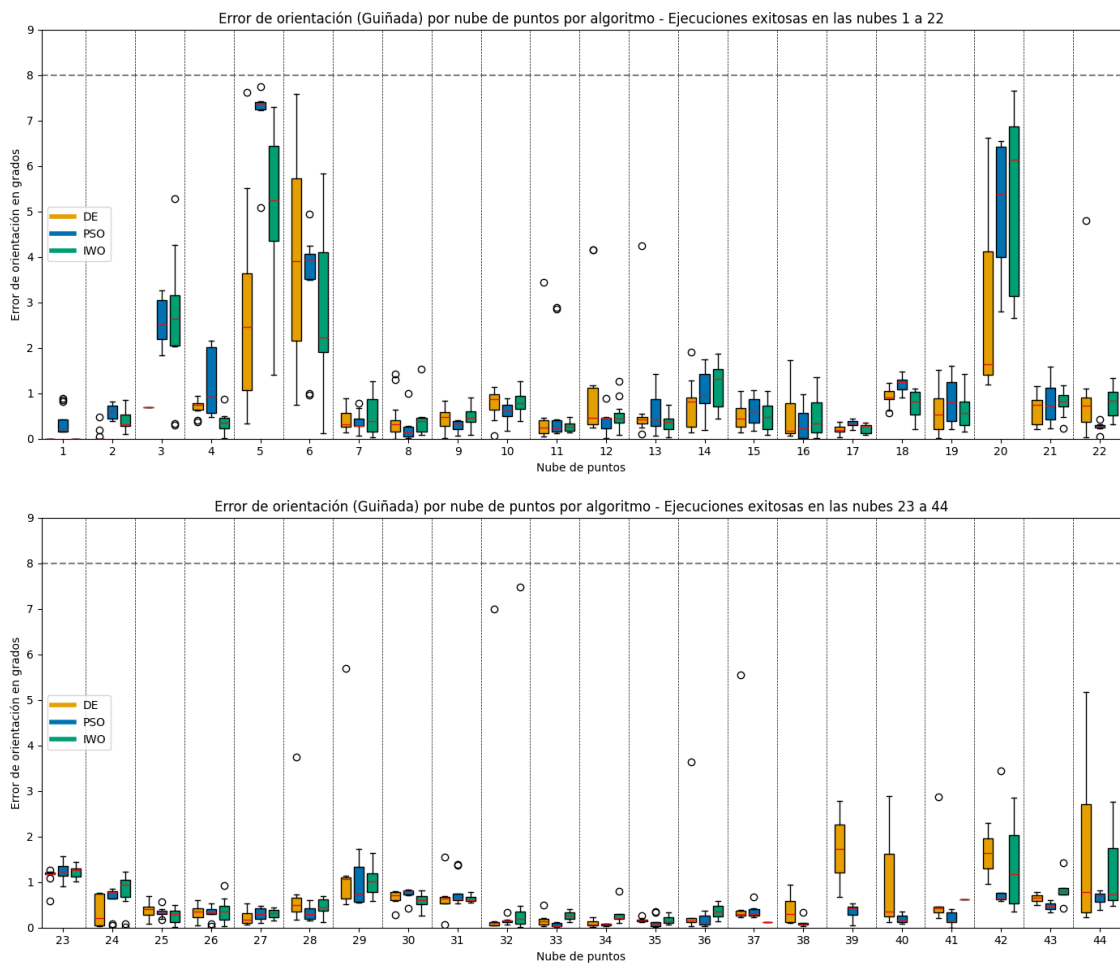


Figura 5.9: Error de orientación en guiñada en ejecuciones exitosas por nube de puntos respecto al umbral de convergencia (línea punteada).

5.2. Resultados de simulación

El objetivo principal de las pruebas de simulación es verificar el correcto funcionamiento de la herramienta desarrollada en ROS2 a bordo de un robot. Además, estas pruebas permitirán estudiar el comportamiento de los algoritmos en un entorno más exigente y comparar los resultados con los obtenidos utilizando el conjunto de datos principal.

Para los análisis en el entorno de simulación se utilizó un escenario que supone un mayor reto para los algoritmos: una pequeña casa poco amueblada y con habitaciones simétricas. La casa se compone de 4 habitaciones rectangulares de dimensiones similares y dos salones cuadrados del mismo tamaño conectados por un pasillo. El salón principal cuenta con una mesa grande en el centro y una estantería pegada a la pared norte. Por otra parte, el salón secundario incluye una mesa de menor tamaño en el centro y un contenedor cilíndrico en la esquina noroeste. Las habitaciones rectangulares cuentan con uno o un par de muebles que sirven de elementos característicos para facilitar la autolocalización del robot.

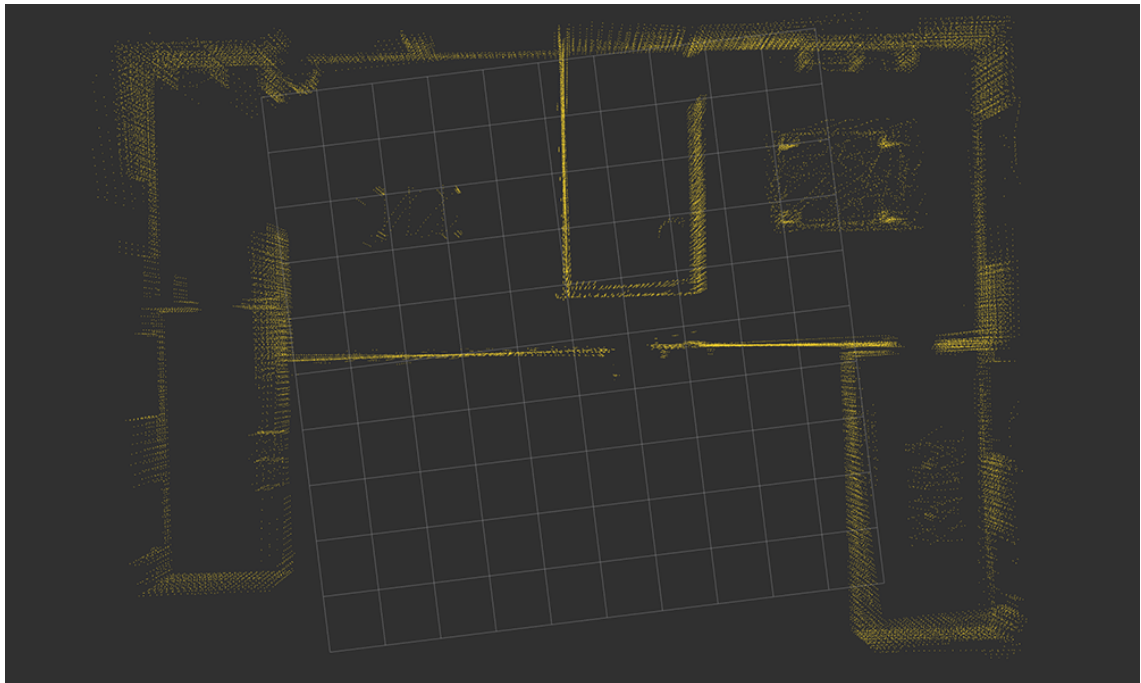


Figura 5.10: Nube de puntos global del entorno de simulación

La nube de puntos global utilizada cuenta con 34.399 puntos y cada una de las nubes

locales con alrededor de 3000. Para realizar las pruebas cada algoritmo se ejecutó en cuatro puntos diferentes del entorno entre diez y veinte veces.

- **Ubicación 1:** El robot se encuentra debajo de la mesa del salón principal. Desde esta posición se pueden ver diversos elementos característicos como las patas de la mesa, el mueble al norte de la habitación y parte del pasillo que conecta ambos salones.
- **Ubicación 2:** El robot se sitúa en la esquina noreste del salón secundario. Desde esta ubicación se observa la mesa en el centro de la habitación, un contenedor cilíndrico y parte de una habitación rectangular.
- **Ubicación 3:** El robot se encuentra en el centro de la habitación rectangular adyacente al salón secundario. Esta habitación cuenta únicamente con dos muebles que dan textura al entorno.
- **Ubicación 4:** El robot se encuentra en la esquina sudeste del salón secundario, delante del pasillo que conecta este con el salón principal.

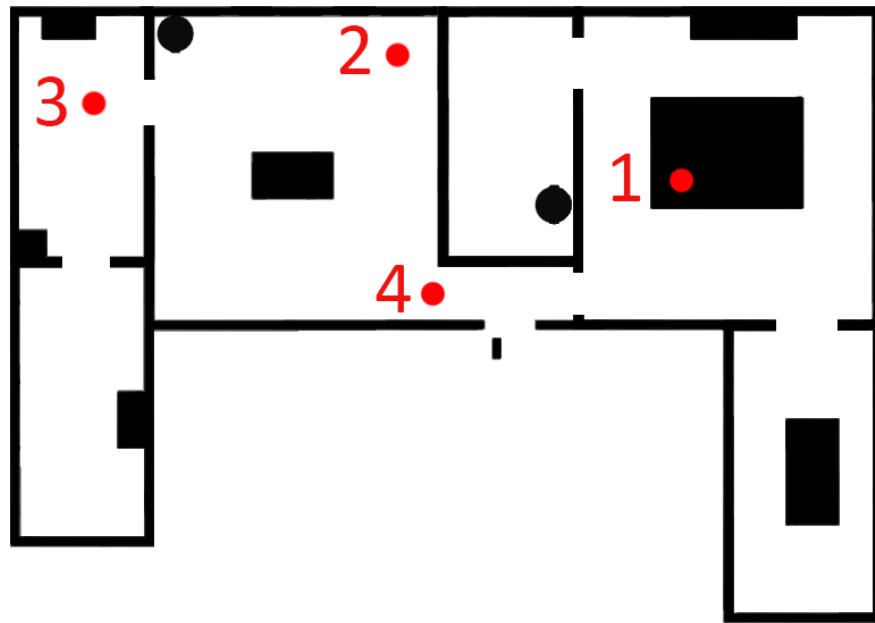


Figura 5.11: Posiciones elegidas para las pruebas en el entorno de simulación

5.2.1. Dificultad del escenario de simulación

El escenario escogido es especialmente complejo para los algoritmos ya que cuenta con un gran número de ubicaciones similares entre sí y un escaso número de elementos distintivos, por lo que los puntos pertenecientes a elementos característicos dentro de cada nube son minoría. Además, el entorno es considerablemente más grande que el utilizado en las pruebas anteriores pero su mapa global cuenta con 11.707 puntos menos.

En la primera ubicación seleccionada las patas de la mesa sirven como elementos característicos y facilitan la autolocalización. Sin embargo, la mesa presenta una simetría longitudinal, lo que dificulta discernir la orientación real en guiñada. Además, en el salón secundario existe otra mesa de características similares, por lo que se puede cometer un error en posición al confundir las mesas de cada salón.

La segunda nube presenta una dificultad mayor ya que, debido a la ubicación del robot, la mayoría de puntos recogidos pertenecen a las dos paredes que conforman la esquina en la que se encuentra. Cómo se cuenta con menos puntos pertenecientes a los elementos distintivos del entorno, como la mesa o el contenedor cilíndrico, el peso que tendrán estos elementos al calcular el error será menor. Si algún candidato encuentra otra esquina en el mapa que coincida en dimensiones, será muy difícil descartarlo antes de converger en un mínimo local ya que la mayoría de puntos de ambas nubes coincidirán.

La tercera nube presenta dificultades por el mismo motivo que la anterior. Aunque esta vez la mayoría de puntos se distribuyen en cuatro paredes en vez de en dos, existen cuatro habitaciones con las mismas dimensiones que se diferencian únicamente por los pocos puntos que conforman los muebles que contienen.

Por último, la cuarta nube, al haber capturado parte del pasillo y el salón con sus muebles, presenta menos dificultades. Aunque dentro de la casa no existen muchas ubicaciones con características similares como en los casos anteriores, sí existe una extremadamente similar, la que se obtiene al situar al robot al otro lado del pasillo, en el salón principal, girado 180 grados.

5.2.2. Porcentaje de convergencia en simulación

De las pruebas realizadas en los cuatro escenarios, DE no consiguió una tasa de convergencia superior al 80 % en ninguno de ellos, mientras que PSO e IWO consiguieron una tasa de éxito superior al umbral únicamente en el último escenario.

Los porcentajes de éxito en convergencia son consistentes con los obtenidos durante las pruebas con el conjunto de datos principal: IWO muestra un comportamiento polarizado, obteniendo una tasa de éxito del 100 % en ciertas situaciones pero tasas inferiores al 20 % en otras. PSO y DE en cambio, aunque no son capaces de converger en la totalidad de sus ejecuciones en ninguna situación, presentan un mayor porcentaje de éxito de media, siendo superior el obtenido por PSO.

Es importante remarcar que la única diferencia respecto a las pruebas realizadas con el conjunto de datos principal está en la forma de obtener la nube de puntos local, que es publicada por el robot en un *topic* de ROS2 al que el nodo se suscribe. El bajo porcentaje de convergencia no se debe a una incorrecta implementación de los algoritmos sino a la dificultad del escenario escogido para la realización de las pruebas. Esta prueba recrea el caso más difícil al que se pueden enfrentar los algoritmos, pues el robot no puede moverse ni girar y se debe determinar con precisión la pose del robot dentro de un entorno con un número muy reducido de elementos distintivos y en un número limitado de iteraciones, utilizando únicamente dos nubes de puntos.

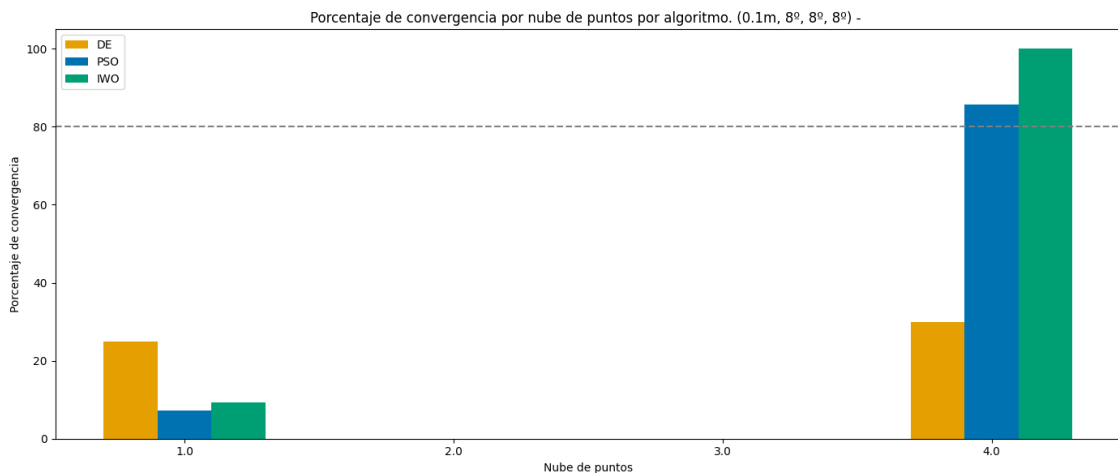


Figura 5.12: Porcentaje de convergencia por nube de puntos en simulación

5.2.3. Tiempo de ejecución y número de iteraciones en simulación

Los tiempos de ejecución y número de iteraciones obtenidos en las ejecuciones exitosas durante las pruebas de simulación son similares a los obtenidos mediante las pruebas con el conjunto de datos principal.

El algoritmo DE fue el más rápido de todos, deteniéndose en la iteración 50 en todas su ejecuciones con un tiempo de ejecución medio de 218 segundos.

El algoritmo IWO volvió a ser el más lento de los tres, con 426 iteraciones de media y un tiempo de ejecución de 2454 segundos.

El algoritmo PSO se situó nuevamente en un término medio entre DE e IWO, con 129 iteraciones y un tiempo de ejecución promedio de 750 segundos.

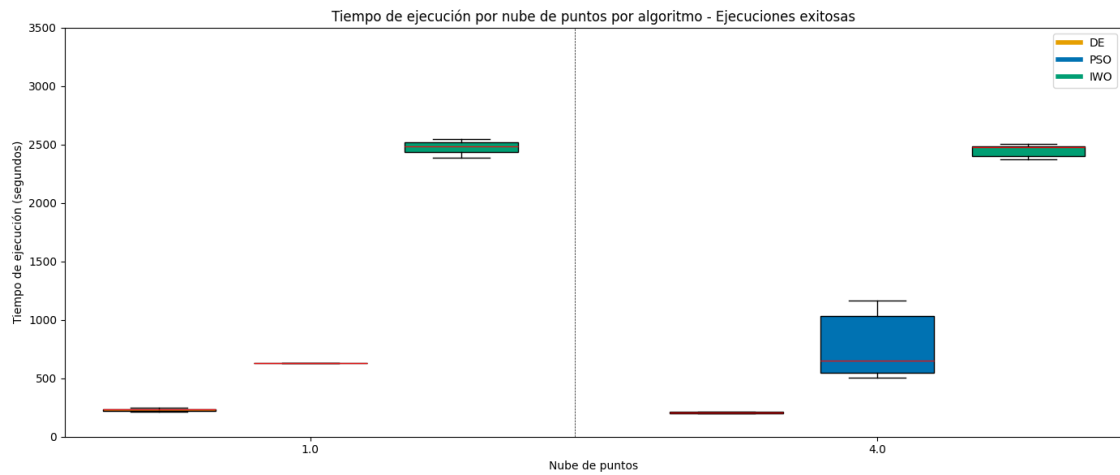


Figura 5.13: Tiempo de ejecución por nube de puntos en simulación

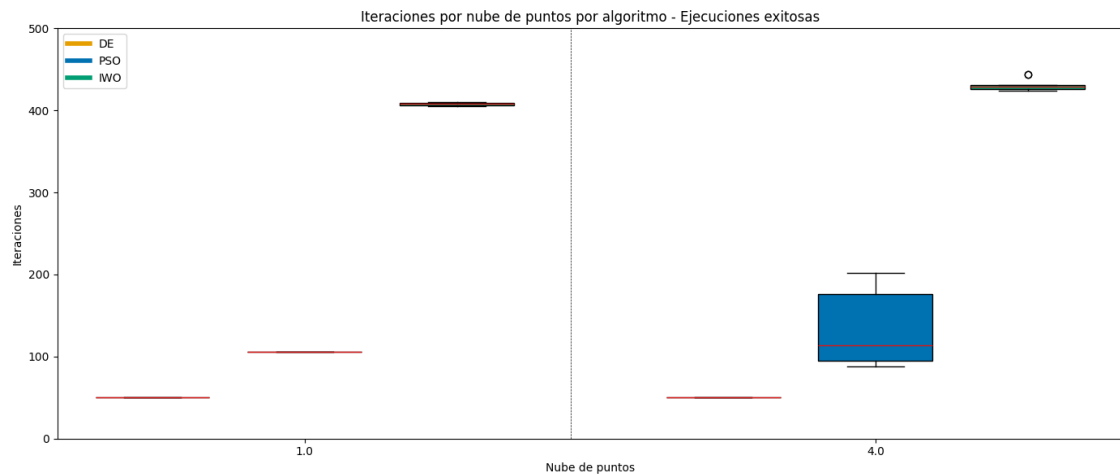


Figura 5.14: Número de iteraciones por nube de puntos en simulación

5.2.4. Errores en posición y orientación en simulación

Error en posición en simulación

Como se observa en la gráfica 5.15, durante las pruebas de simulación ningún algoritmo pudo obtener un error en posición menor a 0,1 metros en las ubicaciones 2 y 3. Sin embargo, en la primera ubicación el error en posición se mantuvo cercano a los 0,1 metros en todas las ejecuciones, mientras que en la cuarta ubicación fue ligeramente menor, entre los 0,06 y los 0,08 metros.

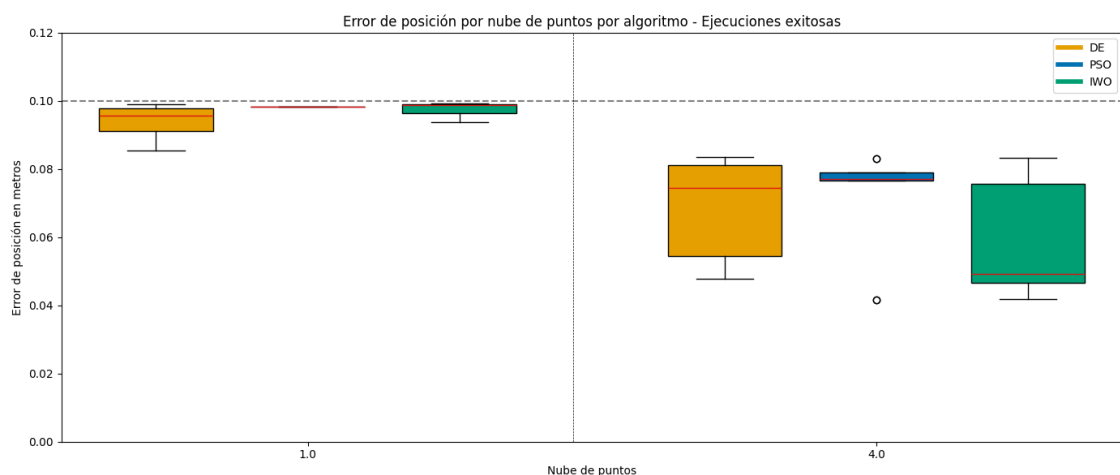


Figura 5.15: Error de posición en ejecuciones exitosas por nube de puntos en simulación respecto al umbral de convergencia (línea punteada)

Errores en cabeceo y alabeo en simulación

Al igual que en las pruebas realizadas con el conjunto de datos principal, los errores en cabeceo y alabeo nunca superaron el umbral establecido de ocho grados de error.

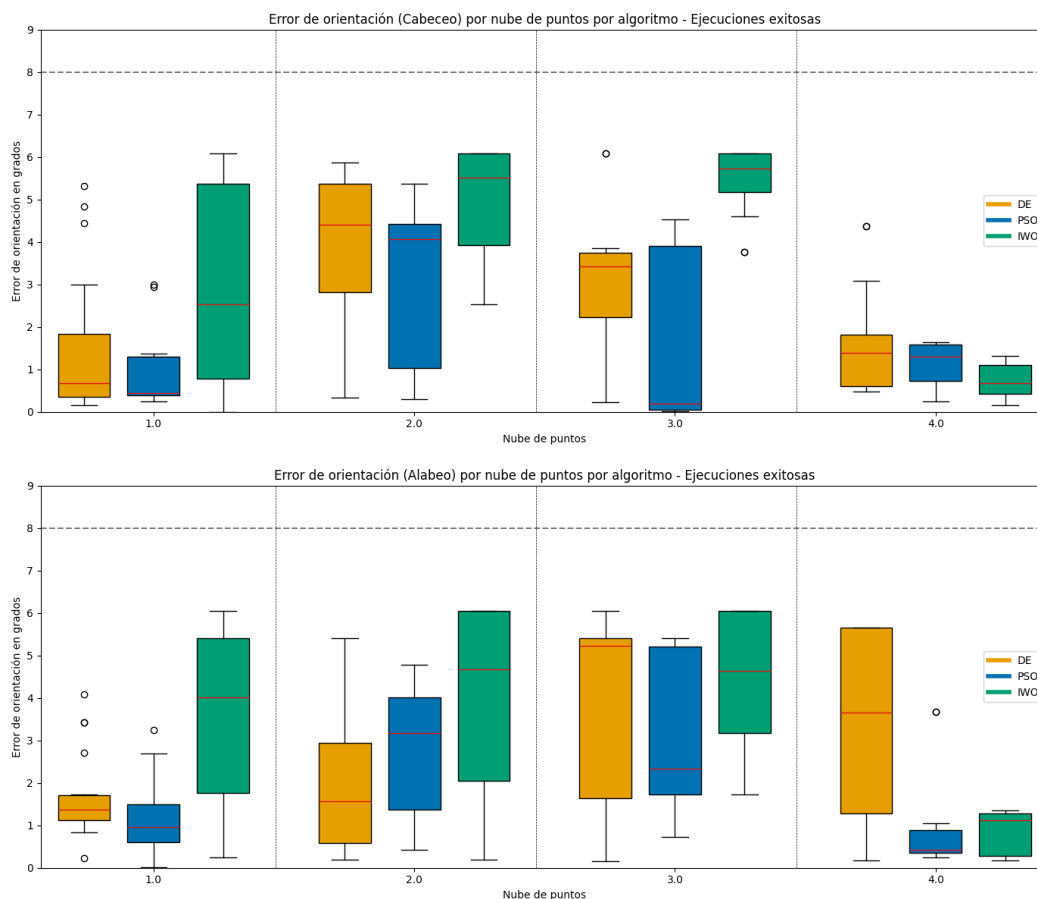


Figura 5.16: Error de orientación en cabeceo y alabeo por nube de puntos en simulación respecto al umbral de convergencia (línea punteada)

Error en guiñada en simulación

Como el escenario está formado exclusivamente por paredes que se unen en ángulos rectos para crear habitaciones cuadradas o rectangulares, los algoritmos tienden a cometer errores en guiñada cercanos a los 90 y 180 grados, los ángulos de simetría rotacional del cuadrado y el rectángulo respectivamente.

En la primera ubicación los tres algoritmos consiguieron converger con un error en guiñada menor a ocho grados en alguna ejecución. DE mostró los mejores resultados, con una media de 0,8 grados, llegando hasta los 2 grados en algunas ejecuciones. PSO

obtuvo errores ligeramente superiores, entre 1 y 3 grados. Por último IWO obtuvo los mayores errores, entre 1 y 6 grados.

En la segunda nube, ningún algoritmo fue capaz de converger con un error menor al umbral.

En la tercera ubicación DE obtuvo el mayor número de ejecuciones con un error menor al umbral. Sin embargo, la media fue significativamente mayor que en la primera nube, con errores comprendidos entre los 5,5 y 7 grados. Por otra parte, PSO obtuvo un error medio de 5 grados, mientras que IWO fue incapaz de converger con un error menor a 8 grados en esta ubicación.

En la cuarta ubicación los tres algoritmos mostraron errores similares, con una media cercana a los 6 grados.

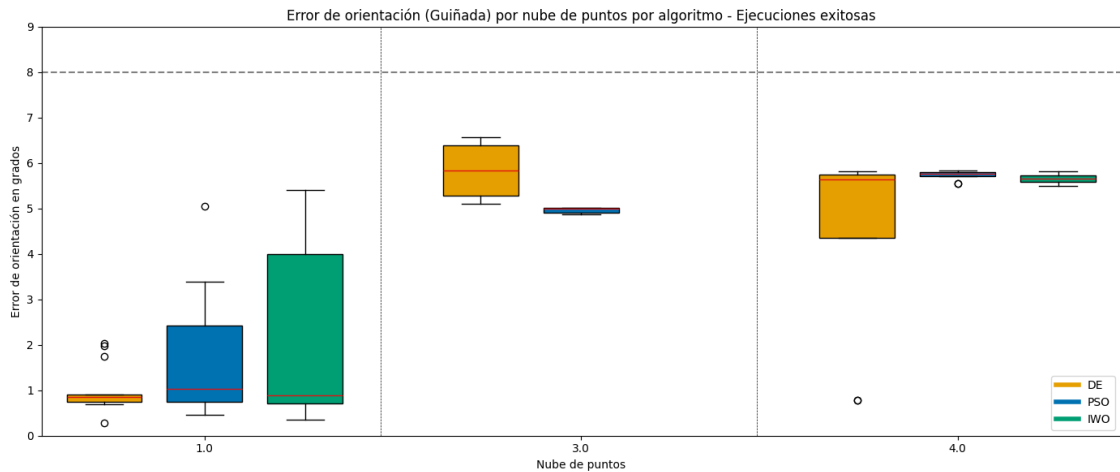


Figura 5.17: Error de orientación en guiñada en ejecuciones exitosas por nube de puntos en simulación respecto al umbral de convergencia (línea punteada)

5.3. Discusión

Debido a la naturaleza heurística de los algoritmos no está garantizado que encuentren la solución óptima. Por ello se debe encontrar un equilibrio entre el coste de ejecución y la calidad del resultado. La implementación del algoritmo escogido debe ser acorde a los requisitos del problema y los parámetros del algoritmo deben ser ajustados

de manera precisa para obtener el resultado deseado.

Por otra parte, es importante que el entorno cuente con un número suficiente de elementos distintivos y la menor simetría posible, además de disponer de un mapa global de calidad y un sensor LiDAR que proporcione una cantidad de puntos suficiente para facilitar la tarea de autolocalización.

Las siguientes tablas resumen las métricas obtenidas en el análisis de los resultados del conjunto de datos principal, remarcando en negrita el mejor resultado en cada categoría.

Algoritmo	Tiempo de ejecución (segundos)	Número de iteraciones	Porcentaje de éxito
DE	183	50	63,63
PSO	599	101	70,45
IWO	1.404	300	52,27

Cuadro 5.1: Resumen de los resultados principales

Algoritmo	Porcentaje de fallo (posición)	Porcentaje de fallo (cabecero)	Porcentaje de fallo (alabeo)	Porcentaje de fallo (guiñada)
DE	30,14	7,69	0,2	17,72
PSO	25,16	0,43	0	10,54
IWO	38,46	0,81	0	22,64

Cuadro 5.2: Resumen de los fallos en convergencia

Aunque los tiempos de ejecución obtenidos son elevados, es importante considerar que estos algoritmos están diseñados para ser ejecutados una única vez. Una vez el robot ha sido localizado, se pueden emplear técnicas de seguimiento, como la propuesta en [Pinto et al., 2015], para estimar la pose del robot. No obstante, si se optara por seguir utilizando estos algoritmos como herramienta de autolocalización única, no sería necesario explorar todo el espacio de búsqueda, lo que reduciría significativamente los tiempos de ejecución.

DE es el algoritmo más sencillo de implementar y el más veloz. Proporciona buenos resultados aunque presenta algunas dificultades para determinar la orientación en escenarios simétricos. Si se ajustan adecuadamente los parámetros y se trabaja en un entorno con suficiente textura puede ser la mejor opción. La sencillez de implementación

junto con el bajo coste computacional requerido lo convierten en un buen candidato para ser implementado a bordo de un robot con baja potencia computacional.

PSO es el algoritmo más robusto y balanceado de los tres. Aunque el tiempo de ejecución es mayor que en el caso de DE, la fiabilidad de los resultados lo convierten en la mejor opción si requiere una autolocalización más precisa. Además, si se dispone de suficiente potencia computacional se podría disminuir el tiempo de ejecución significativamente.

IWO es el algoritmo más lento de los tres y el que presenta mayores errores en posición y en orientación en situaciones difíciles. Sin embargo, demostró ser el más robusto en situaciones sencillas, convergiendo con éxito un 100 % de las veces en el mayor número de escenarios. Si se trabaja en un entorno preparado adecuadamente para el algoritmo en el que se evitan las regiones simétricas y de baja textura y el tiempo de ejecución no es un factor limitante, puede ser la mejor opción.

Aunque los algoritmos mostraron resultados prometedores tras las pruebas con el conjunto de datos principal, el bajo porcentaje de convergencia obtenido mediante las pruebas en simulación sugiere que los algoritmos no están preparados para ser utilizados como herramienta única de autolocalización en entornos de baja textura y alta simetría si el robot carece de la capacidad de moverse o girar mientras se realiza la autolocalización.

Los tres algoritmos han sido sometidos a pruebas rigurosas, en las que se exigía encontrar la pose exacta del robot en un número limitado de iteraciones, con un margen de error de 0,1 metros y 8 grados, dentro de un entorno de varias decenas de metros cuadrados. Para esta tarea, solo se disponía de un mapa global y un mapa local en forma de nubes de puntos, sin posibilidad de mover o girar el robot para obtener más información y refinar la búsqueda. Tras analizar los resultados obtenidos, ninguno de los tres algoritmos ha demostrado una robustez suficiente como para ser utilizado como herramienta única de autolocalización en entornos desafiantes. Sin embargo, si se trabaja en un entorno preparado adecuadamente, podrían ser una opción viable.

Capítulo 6

Conclusiones

En este capítulo se analizará el desarrollo completo del trabajo desarrollado, evaluando el cumplimiento de los objetivos planteados inicialmente. También se estudiará la posibilidad de continuar este trabajo explorando nuevas metodologías y aplicaciones en diferentes contextos. Por último se tratarán las competencias requeridas y adquiridas durante la realización del trabajo.

6.1. Análisis de los objetivos propuestos

El objetivo principal de este trabajo era desarrollar un nodo de ROS2 que implementase tres algoritmos evolutivos y permitiese a un robot autolocalizarse utilizando únicamente un sensor LiDAR. El nodo se utilizaría para recopilar un extenso conjunto de datos que permitiría analizar las fortalezas y debilidades de los distintos algoritmos en varios escenarios. Además, el nodo debía ser lo suficientemente robusto para poder ser utilizado a bordo de un robot simulado y, más adelante, sobre un robot real.

Mediante las pruebas de simulación, el nodo ha demostrado funcionar correctamente y ser lo suficientemente robusto como para ser implementado a bordo de un robot real. Por otra parte, los resultados obtenidos han sido abundantes y de calidad, lo que ha permitido estudiar cada algoritmo en profundidad y determinar sus limitaciones al utilizarse como herramienta única de localización en un entorno desafiante.

En resumen, se ha conseguido crear la herramienta propuesta y realizar un análisis detallado de los algoritmos, cumpliendo así con la totalidad de los objetivos propuestos.

6.2. Líneas de trabajos futuros

La herramienta desarrollada podría combinarse con técnicas de odometría para generar resultados más precisos utilizando múltiples nubes de puntos y una

aproximación recursiva. Tras ejecutar el algoritmo una primera vez, el robot giraría o avanzaría una distancia conocida y se volvería a ejecutar el algoritmo, esta vez utilizando como entrada la nube de puntos calculada en la ejecución anterior desplazada la misma distancia que se haya movido el robot. El proceso se repetiría hasta que los errores cumplan con el umbral establecido. Este mecanismo permitiría filtrar los mínimos locales, mejorando significativamente la calidad de los resultados. Sin embargo, también incrementaría el tiempo de ejecución, ya que el algoritmo necesitaría ejecutarse varias veces.

Como continuación del análisis realizado, se podría estudiar en mayor profundidad la robustez de cada algoritmo ejecutándolo en escenarios ruidosos o con nubes de puntos incompletas. Por ejemplo, se podría utilizar el mapa global de un almacén vacío y ejecutar los algoritmos sobre el robot una vez el almacén haya sido abastecido. Para analizar la capacidad de convergencia de cada algoritmo sin tener en cuenta limitaciones de tiempo o coste computacional se podría incrementar el número máximo de iteraciones así como el tamaño máximo de la población y establecer condiciones de parada más exigentes. Un aumento en el tamaño de la población mejoraría notablemente la precisión en los resultados, aunque también aumentaría el tiempo de ejecución.

6.3. Competencias

El desarrollo de este trabajo ha involucrado un extenso conjunto de conocimientos generales propios de las ingenierías software, así como conocimientos específicos del campo de la robótica. Como base fundamental para el trabajo, se requería el dominio de los lenguajes de programación Python y MATLAB, así como de ROS2, el uso de sistemas operativos basados en Linux, simuladores robóticos y la teoría de autolocalización, conocimientos adquiridos durante la realización del grado en Ingeniería Robótica Software.

Por otro lado, la realización de este trabajo ha permitido la adquisición de nuevos conocimientos relativos al campo de la inteligencia artificial, centrados principalmente en los algoritmos metaheurísticos bioinspirados. Además del desarrollo de habilidades de investigación, toma de datos y análisis estadístico de los mismos.

Bibliografía

- [Belter and Nowicki, 2019] Belter, D. and Nowicki, M. R. (2019). Optimization-based legged odometry and sensor fusion for legged robot continuous localization. *Robotics and Autonomous Systems*, 111:110–124.
- [Carballeira López, 2022] Carballeira López, J. (2022). *Global Localization based on Evolutionary Optimization Algorithms for Indoor and Underground Environments*. Doctoral thesis on computer science and artificial intelligence, University Carlos III of Madrid.
- [Dastjerdi et al., 2016] Dastjerdi, H. V., Menhaj, M. B., and Shataei, S. (2016). Self-localization of humanoid robots using particle swarm optimization algorithm. In *2016 Artificial Intelligence and Robotics (IRANOPEN)*, pages 98–102. IEEE.
- [Font and Batlle, 2006] Font, J. M. and Batlle, J. A. (2006). Mobile robot localization. revisiting the triangulation methods. *IFAC Proceedings Volumes*, 39(15):340–345. 8th IFAC Symposium on Robot Control.
- [Havangi, 2019] Havangi, R. (2019). Mobile robot localization based on pso estimator. *Asian Journal of Control*, 21(4):2167–2178.
- [Havangi et al., 2010] Havangi, R., Nekoui, M. A., and Teshnehlab, M. (2010). A multi swarm particle filter for mobile robot localization. *International Journal of Computer Science*, 7(3):15–22.
- [Huang and Dissanayake, 2016] Huang, S. and Dissanayake, G. (2016). *Robot Localization: An Introduction*, pages 1–10. John Wiley and Sons, Ltd.
- [Huo et al., 2013] Huo, J., Ma, L., Yu, Y., and Wang, J. (2013). Hybrid algorithm based mobile robot localization using de and pso. In *Proceedings of the 32nd Chinese Control Conference*, pages 5955–5959. IEEE.

- [Li et al., 2024a] Li, J., Soradi-Zeid, S., Yousefpour, A., and Pan, D. (2024a). Improved differential evolution algorithm based convolutional neural network for emotional analysis of music data. *Applied Soft Computing*, 153:111262.
- [Li et al., 2024b] Li, L., Schulze, L., and Kalavadia, K. (2024b). Promising slam methods for automated guided vehicles and autonomous mobile robots. *Procedia Computer Science*, 232:2867–2874. 5th International Conference on Industry 4.0 and Smart Manufacturing (ISM 2023).
- [Mehrabian and Lucas, 2006a] Mehrabian, A. and Lucas, C. (2006a). A novel numerical optimization algorithm inspired from weed colonization. *Ecological Informatics*, 1(4):355–366.
- [Mehrabian and Lucas, 2006b] Mehrabian, A. and Lucas, C. (2006b). A novel numerical optimization algorithm inspired from weed colonization. *Ecological Informatics*, 1:355–366.
- [Moreno et al., 2002] Moreno, L., Armingol, J. M., Garrido, S., De La Escalera, A., and Salichs, M. A. (2002). A genetic algorithm for mobile robot localization using ultrasonic sensors. *Journal of Intelligent and Robotic Systems*, 34:135–154.
- [Negenborn, 2003] Negenborn, R. (2003). Robot localization and kalman filters. *Utrecht Univ., Utrecht, Netherlands, Master’s thesis INF/SCR-0309*.
- [Nemec et al., 2019] Nemec, D., Šimák, V., Janota, A., Hruboš, M., and Bubeníková, E. (2019). Precise localization of the mobile wheeled robot using sensor fusion of odometry, visual artificial landmarks and inertial sensors. *Robotics and Autonomous Systems*, 112:168–177.
- [Panigrahi and Bisoy, 2022] Panigrahi, P. K. and Bisoy, S. K. (2022). Localization strategies for autonomous mobile robots: A review. *Journal of King Saud University - Computer and Information Sciences*, 34(8, Part B):6019–6039.
- [Pinto et al., 2015] Pinto, A. M., Moreira, A. P., and Costa, P. G. (2015). A localization method based on map-matching and particle swarm optimization. *Journal of Intelligent & Robotic Systems*, 77:313–326.
- [Rajurkar et al., 2016] Rajurkar, S. D., Kar, A. K., Goswami, S., and Verma, N. K. (2016). Optimal path estimation and tracking for an automated vehicle using ga optimized fuzzy controller. In *2016 11th International Conference on Industrial and Information Systems (ICIIS)*, pages 365–370. IEEE.

- [Salhi et al., 2019] Salhi, I., Poreba, M., Piriou, E., Gouet-Brunet, V., and Ojail, M. (2019). *Chapter 8 - Multimodal Localization for Embedded Systems: A Survey*, pages 199–278. Academic Press.
- [Steffen, 2022] Steffen, V. (2022). Particle swarm optimization with a simplex strategy to avoid getting stuck on local optimum. *AI, Computer Science and Robotics Technology*.
- [Sur and Shukla, 2013] Sur, C. and Shukla, A. (2013). Discrete invasive weed optimization algorithm for graph based combinatorial road network management problem. In *2013 International Symposium on Computational and Business Intelligence*, pages 254–257.
- [Vahdat et al., 2007] Vahdat, A. R., NourAshrafoddin, N., and Ghidary, S. S. (2007). Mobile robot global localization using differential evolution and particle swarm optimization. In *2007 IEEE congress on evolutionary computation*, pages 1527–1534. IEEE.
- [Wang et al., 2015] Wang, H., Liu, W., Zhang, F., Yang, S. X., and Zhang, L. (2015). A ga-fuzzy logic based extended kalman filter for mobile robot localization. In *2015 12th International Conference on Fuzzy Systems and Knowledge Discovery (FSKD)*, pages 319–323. IEEE.
- [Yang et al., 2007] Yang, Z., Tang, K., and Yao, X. (2007). Differential evolution for high-dimensional function optimization. In *2007 IEEE Congress on Evolutionary Computation*, pages 3523–3530.
- [Yarovoi and Cho, 2024] Yarovoi, A. and Cho, Y. K. (2024). Review of simultaneous localization and mapping (slam) for construction robotics applications. *Automation in Construction*, 162:105344.